



第5章 中央处理器II





5.5 硬布线控制器



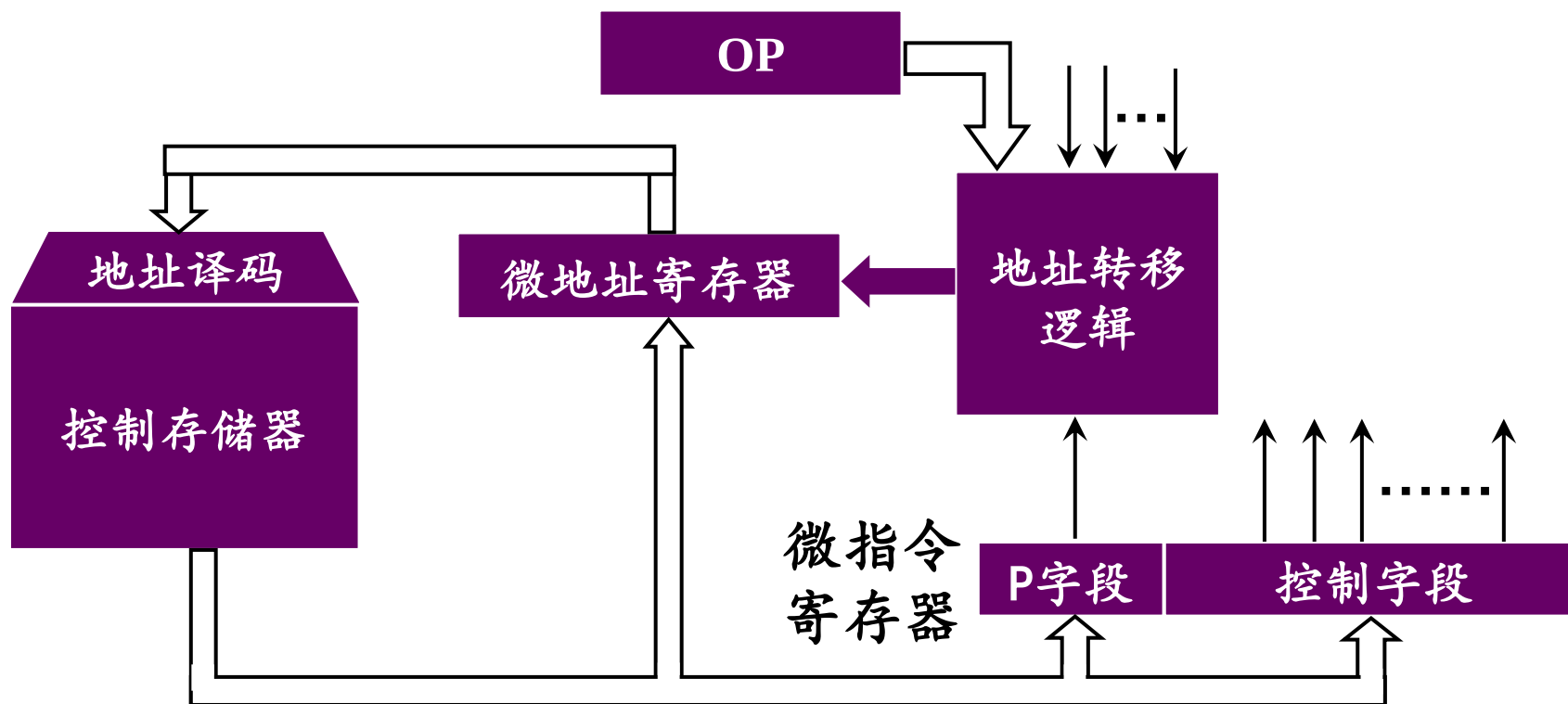


基本思想

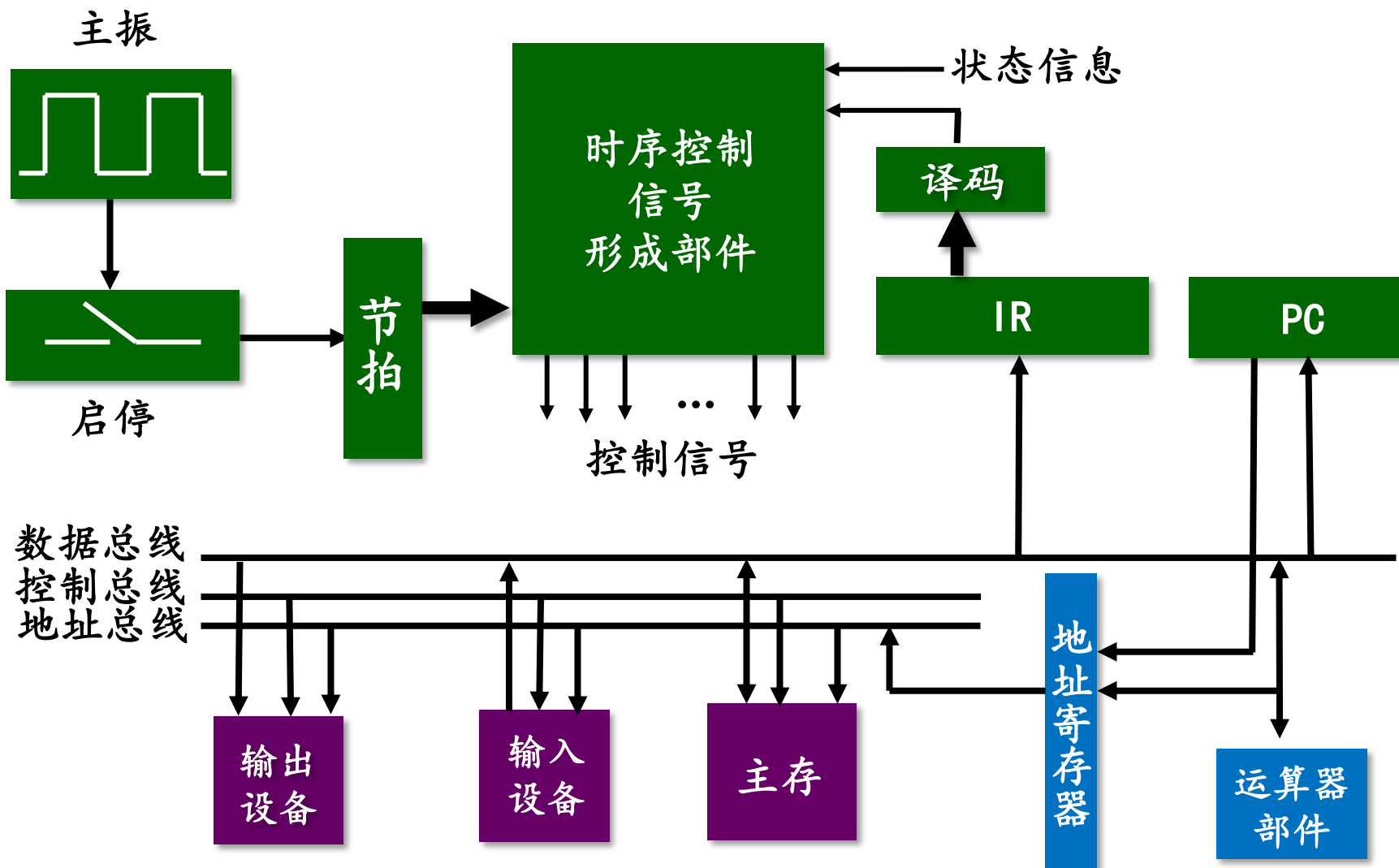
- 硬布线控制器（也称为组合逻辑控制器）是早期设计计算机的一种方法之一，也是RISC计算机和高性能计算机普遍采用的设计方法
- 控制部件为产生固定时序控制信号的逻辑电路
- 特点
 - ◆ 形成控制信号的传输延迟时间少，能提高系统的执行速度
 - ◆ 设计控制CPU各功能部件运行所需要的时序控制信号的逻辑比较复杂
 - ◆ 设计变更比较困难
 - ◆ 与微程序控制相比，硬布线控制的速度较快。因为微程序控制中每条微指令都要从控存中读取，影响了速度；而硬布线控制主要取决于电路延迟



回顾 微程序控制器原理框图

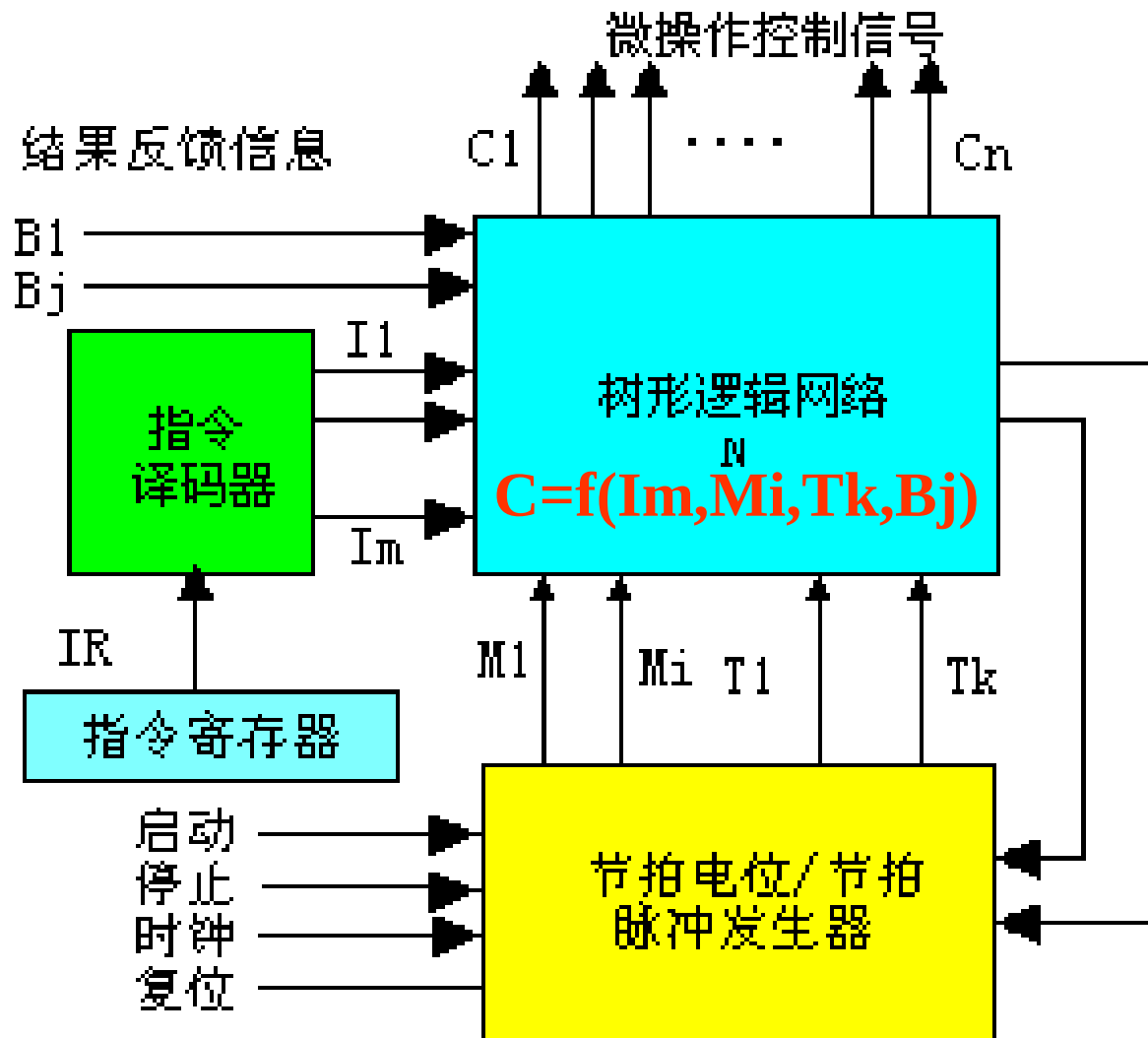


硬布线控制器原理框图



硬布线控制器的结构图

- 逻辑网络的输入信号来源有三个：
- (1)来自指令操码译码器的输出 I_m
- (2)来自执行部件的反馈信息 B_j
- (3)来自时序产生器的时序信号，包括节拍电位信号 M 和节拍脉冲信号 T





指令执行流程

- 在硬布线控制器中，通常时序产生器除了产生节拍脉冲信号外，还应当产生节拍电位信号（而微程序控制器中节拍电位是用微指令周期来体现的）
- 在一个指令周期中要顺序执行一系列微操作，需要设置若干个节拍电位，来保证这些微操作执行的先后顺序

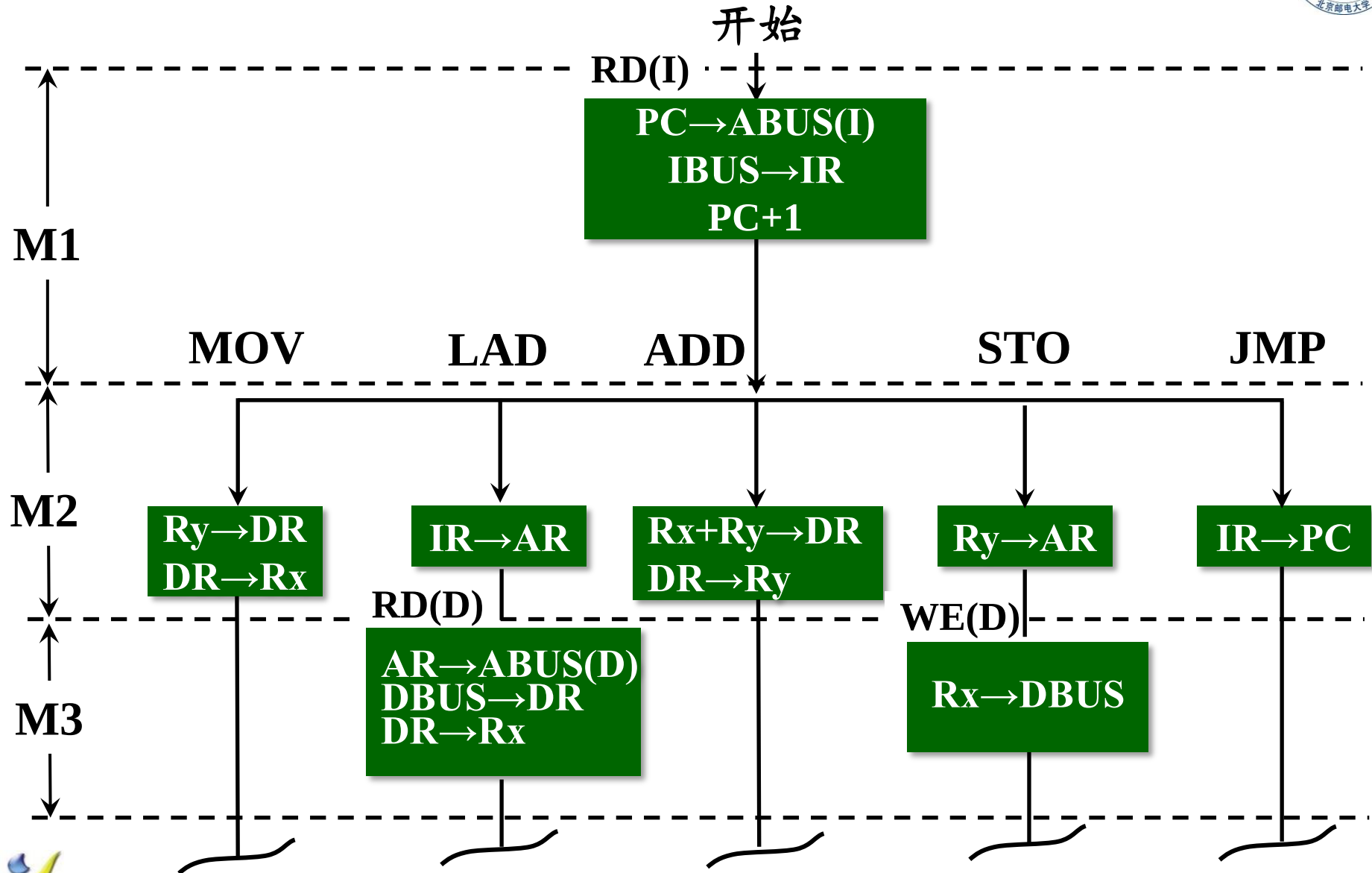
例：若C3为打入程序计数器PC的控制信号，在PC+1或执行JMP时该信号都有效（如下图所示），逻辑表达式：

$$C3 = M1T4 + M2T4JMP$$





硬连线控制器的指令周期流程图





微操作控制信号的产生

- 在微程序控制器中，微操作控制信号由微指令产生，并且可以重复使用
- 在硬布线控制器中，某一微操作控制信号由布尔表达式描述的输出函数产生
- 设计微操作控制信号的方法：
 1. 根据所有机器指令流程图，寻找出产生同一个微操作信号的所有条件
 2. 与适当的节拍电位和节拍脉冲组合，写出其布尔代数表达式并进行简化
 3. 用门电路或可编程器件来实现





例5.3 (1)

根据上图，写出以下操作控制信号RD(I)、RD(D)、WE(D)、LDPC、LDIR、LDAR、LDDR、PC+1、LDRy的逻辑表达式。其中每个操作控制信号的含义：

RD (I) — 指存读命令；

RD (D) — 数存读命令

WE (D) — 数存写命令；

LDPC—打入PC

LDIR—打入IR；

LDAR—打入数存地址寄存器

LDDR—打入数据缓冲寄存器；

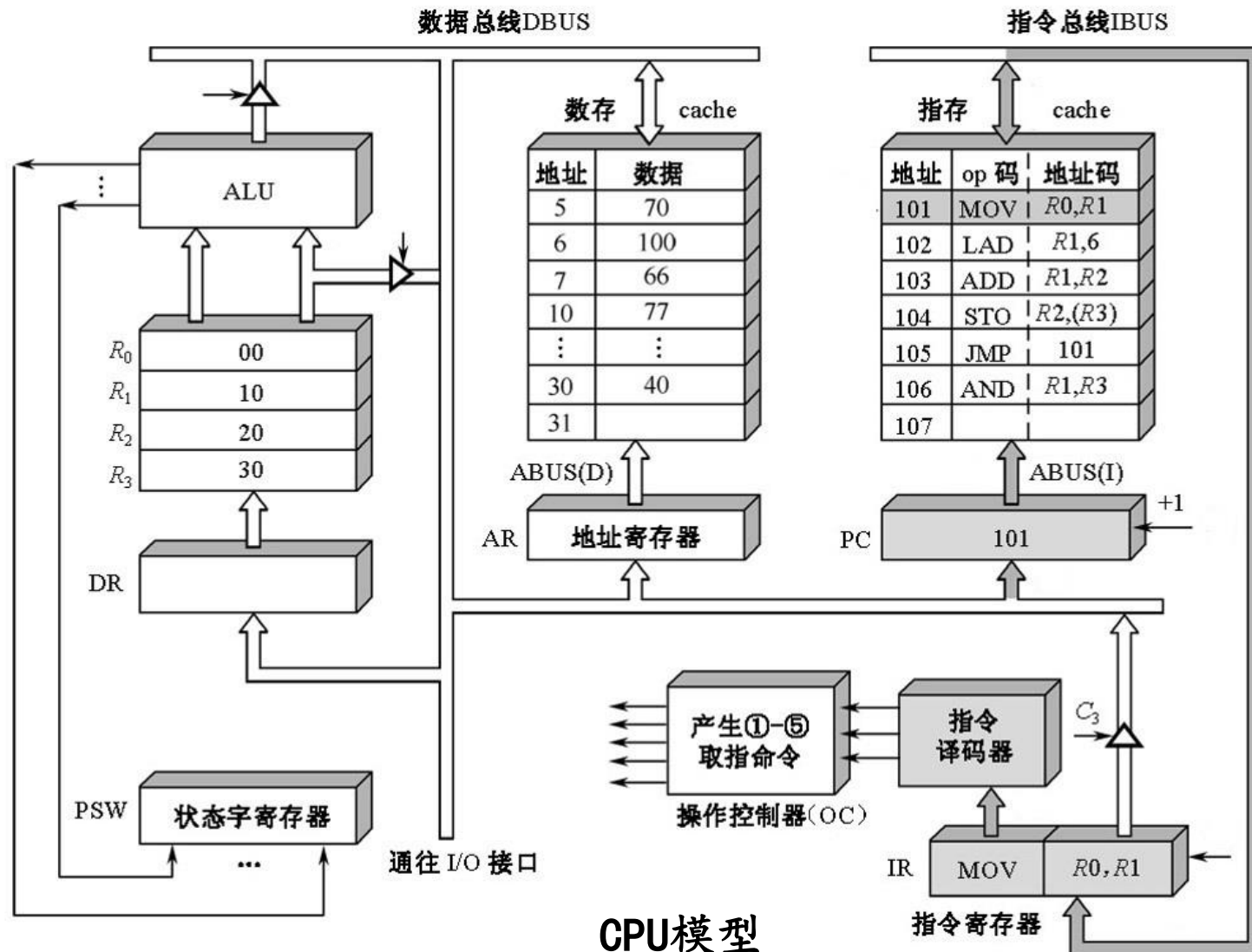
PC+1—PC加1

LDRy—打入Ry寄存器





例5.3 (2)





例5.3 (3)

解：设M1、M2和M3为节拍电位信号，T1、T2、T3和T4为一个CPU周期中的节拍脉冲信号，MOV、LAD、ADD、STO和JMP分别表示对应机器指令操作码的译码输出信号，则有如下逻辑表达式：

$$\text{RD (I)} = \text{M1 (电位信号)}$$

$$\text{RD (D)} = \text{M3LAD (电位信号)}$$

$$\text{WE (D)} = \text{M3T3STO (脉冲信号)}$$

$$\text{LDPC} = \text{M1T4} + \text{M2T4JMP}$$

$$\text{LDIR} = \text{M1T3}$$

$$\text{LDAR} = \text{M2T4 (LAD} + \text{STO)}$$

$$\text{LDDR} = \text{M2T3 (MOV} + \text{ADD)} + \text{M3T3LAD}$$

$$\text{PC} + 1 = \text{M1T3}$$

$$\text{LDRy} = \text{M2T4ADD}$$





动手做

相对于微程序控制器，硬布线控制器的特点是

- A. 指令执行速度慢，指令功能的修改和扩展容易
- B. 指令执行速度慢，指令功能的修改和扩展难
- C. 指令执行速度快，指令功能的修改和扩展容易
- D. 指令执行速度快，指令功能的修改和扩展难



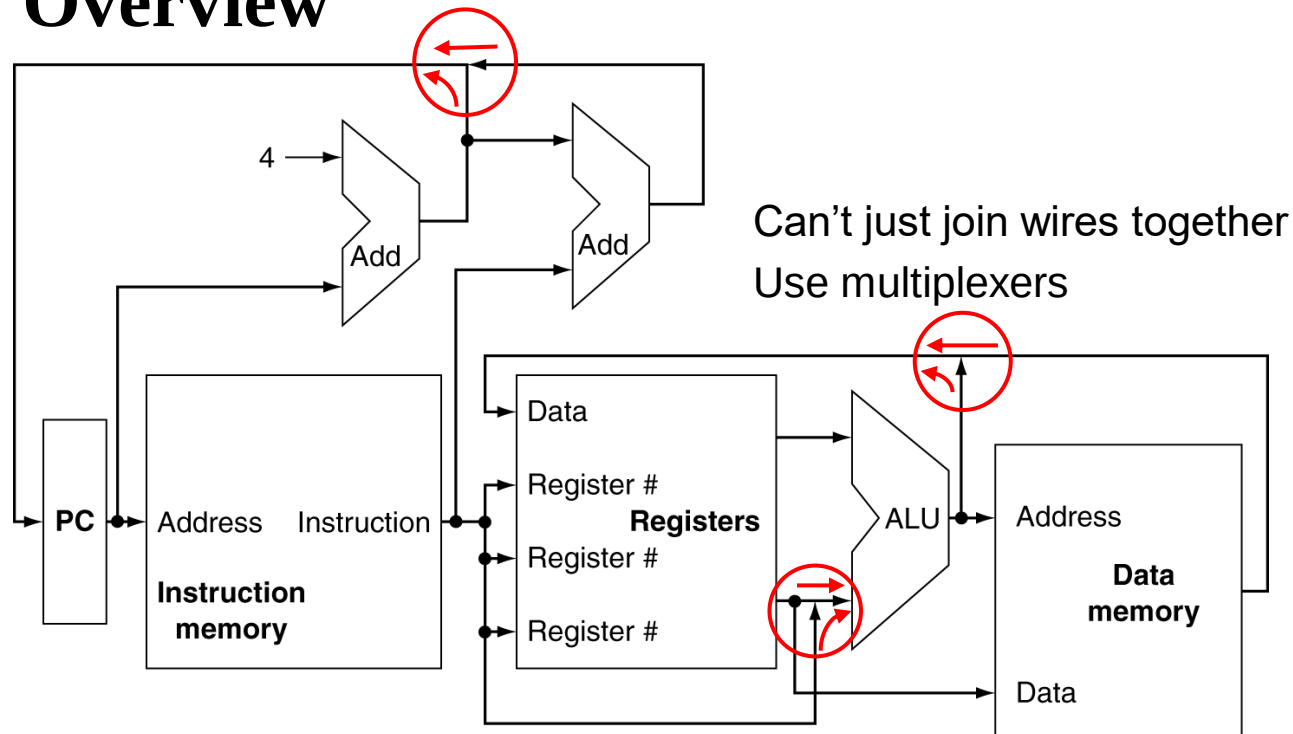


实例 一个基本的MIPS实现

■ Simple subset, shows most aspects

- ◆ Memory reference: lw, sw
- ◆ Arithmetic/logical: add, sub, and, or, slt (小于则设置)
- ◆ Control transfer: beq, j

■ CPU Overview

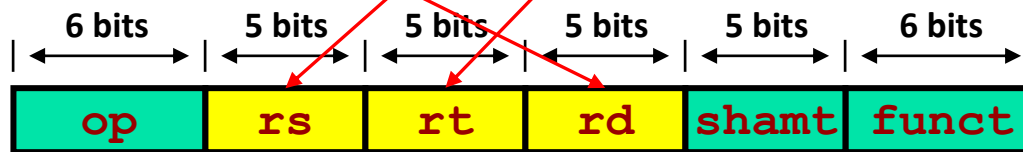




MIPS指令格式

- 算术逻辑指令: add, sub, and, or, slt
- 取数/存数指令: lw, sw
- 控制流指令: beq, j

add \$t0, \$s1, \$s2



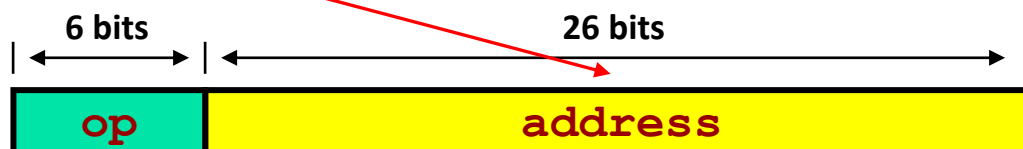
R-Format

lw \$t0, 1002 (\$s2) beq \$t0, \$t1, Lable



I-Format

j Lable



J-Format





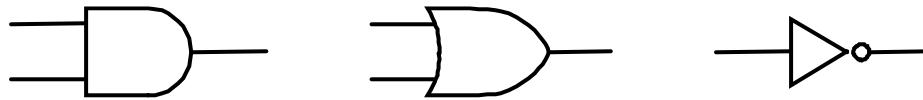
Two Different Logic Elements

■ Combinational element, e.g., ALU

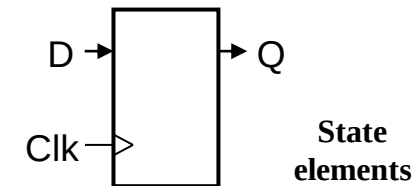
- ◆ Operate on data
- ◆ Output is a function of input

■ State (sequential) elements, e.g., register

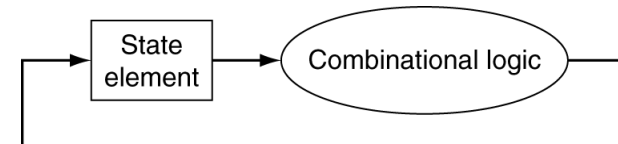
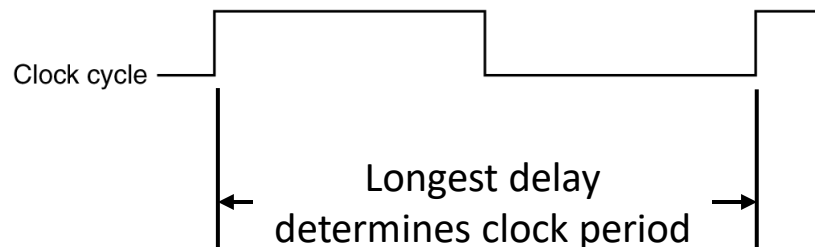
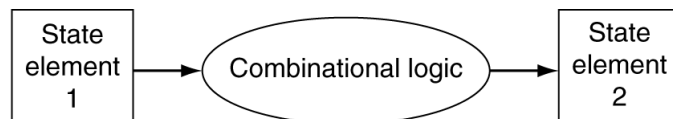
- ◆ Store information



Combinational elements



State elements





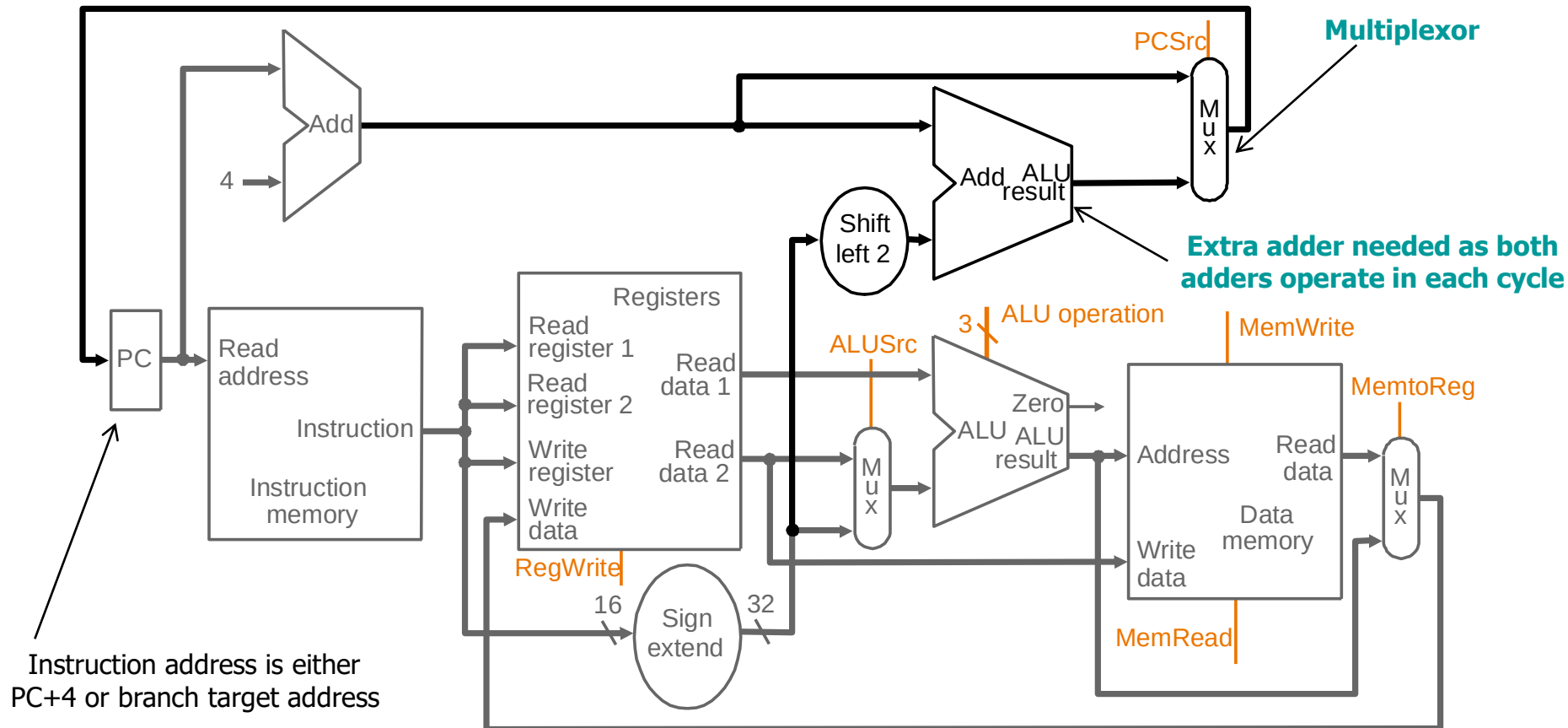
Single-cycle Implementation of MIPS

单周期

- Our first implementation of MIPS will use a **single** long clock **cycle** for every instruction
- Every instruction begins on one up (or, down) clock edge and ends on the next up (or, down) clock edge
- This approach is *not practical* as it is much slower than a **multicycle** implementation where different instruction classes can take different numbers of cycles
 - ◆ in a single-cycle implementation every instruction must take the same amount of time as the slowest instruction
- Even though the single-cycle approach is not practical it is simple and useful to understand first
- *Note* : we shall implement **jump** at the very end



MIPS Datapath: Single-Cycle



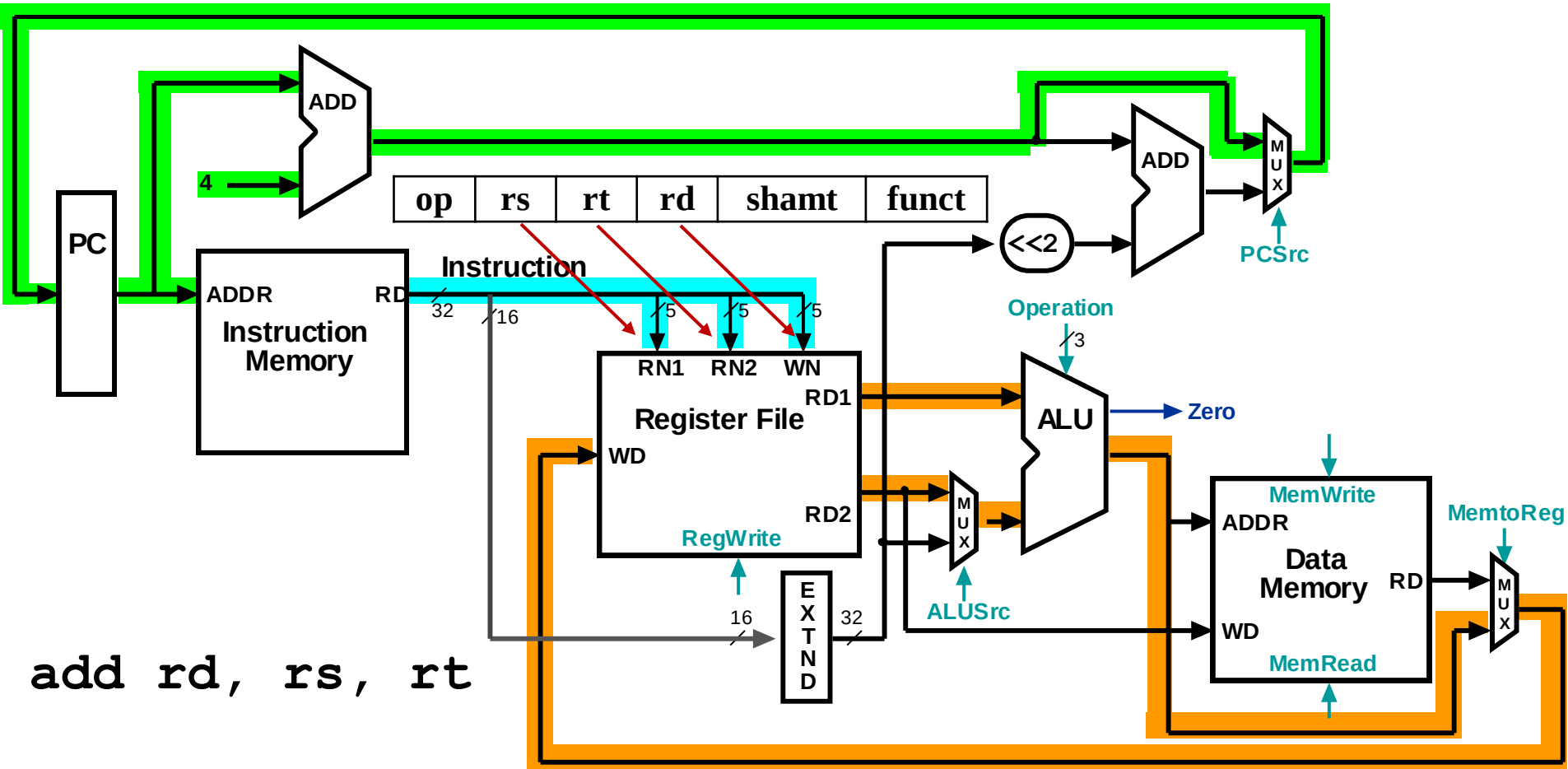
Adding branch capability and another multiplexor

Important note: in a single-cycle implementation data cannot be stored during an instruction – it only moves through combinational logic

橙色表示控制信号

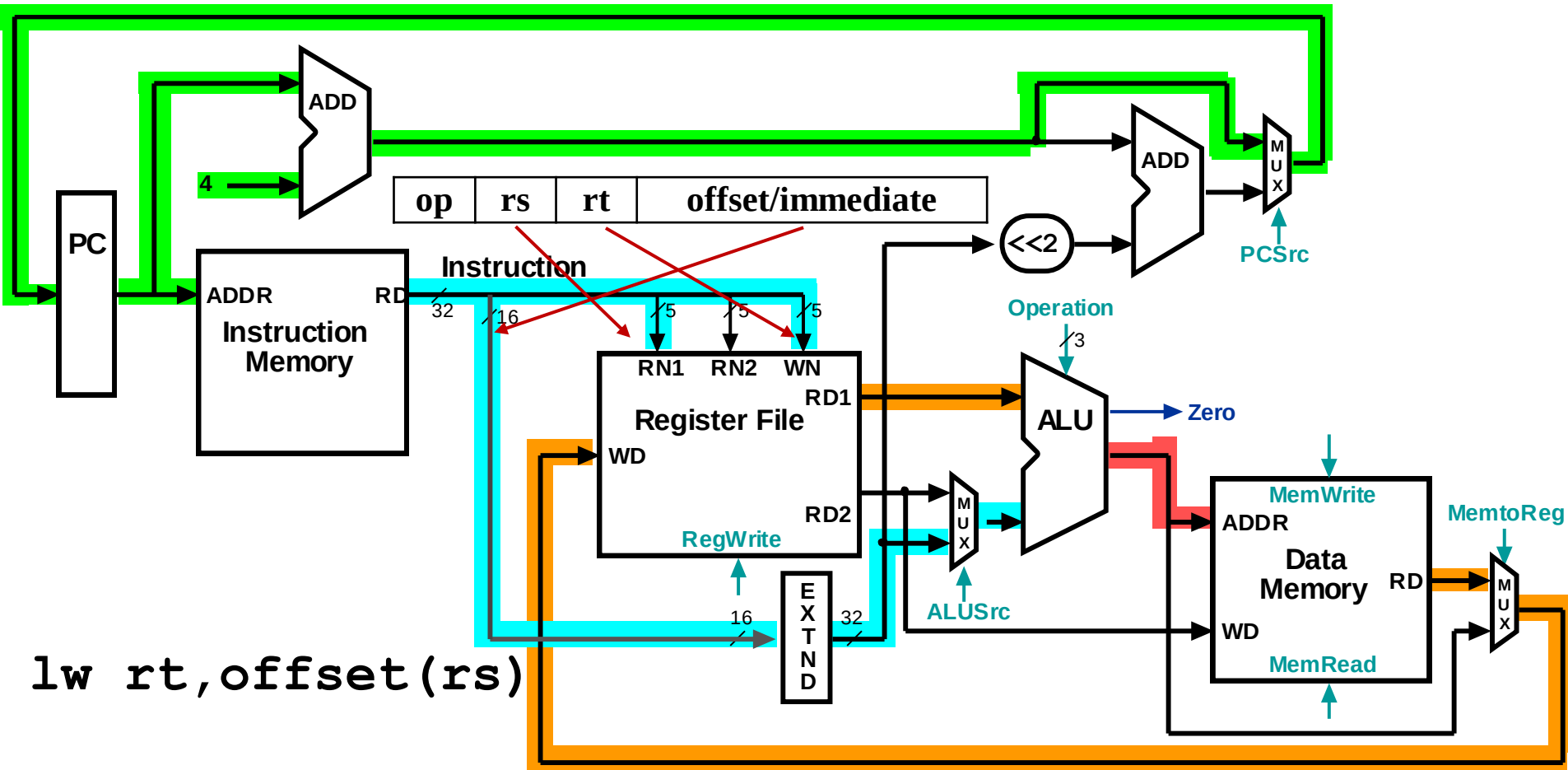


Datapath Executing add



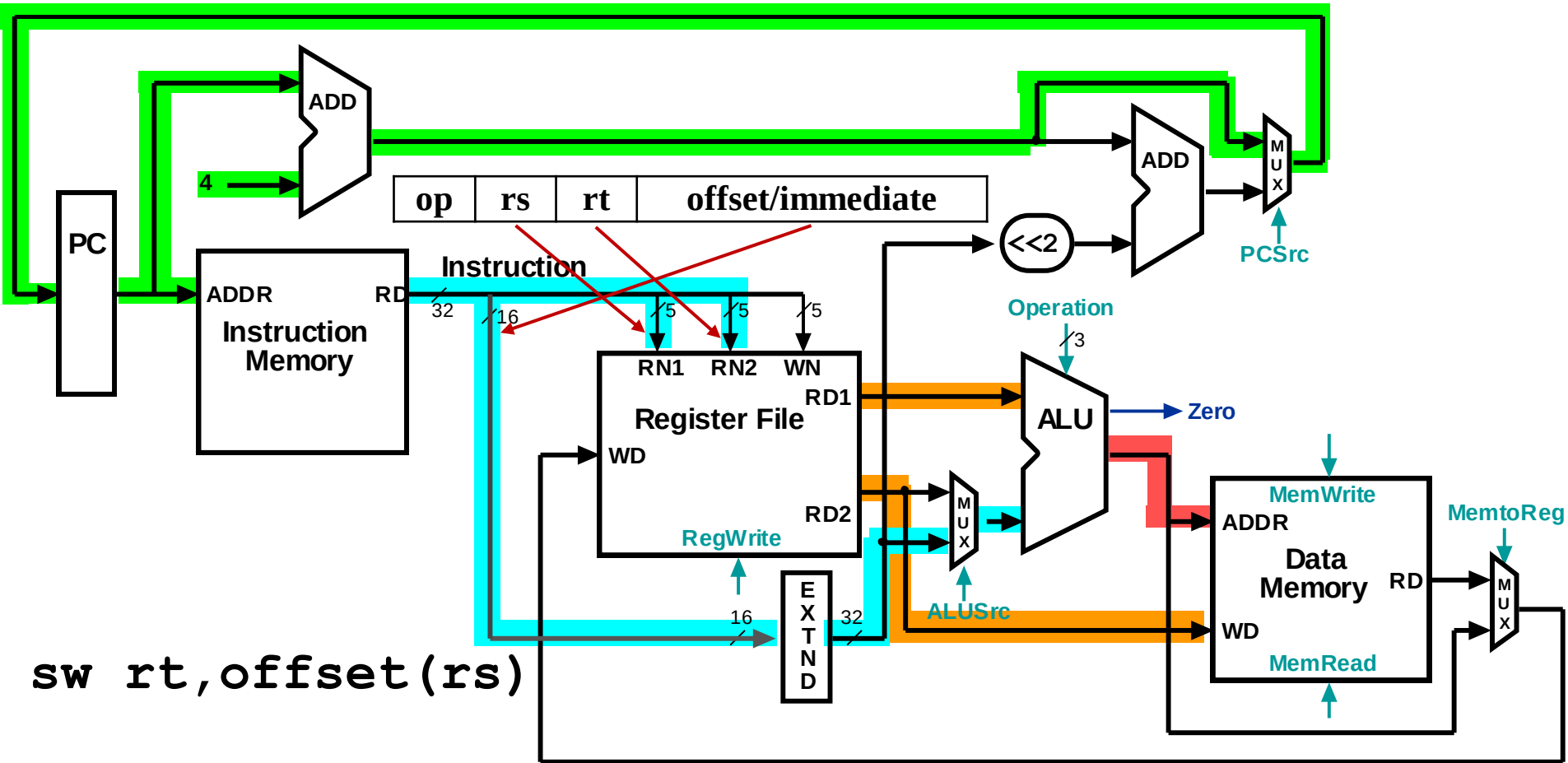


Datapath Executing lw



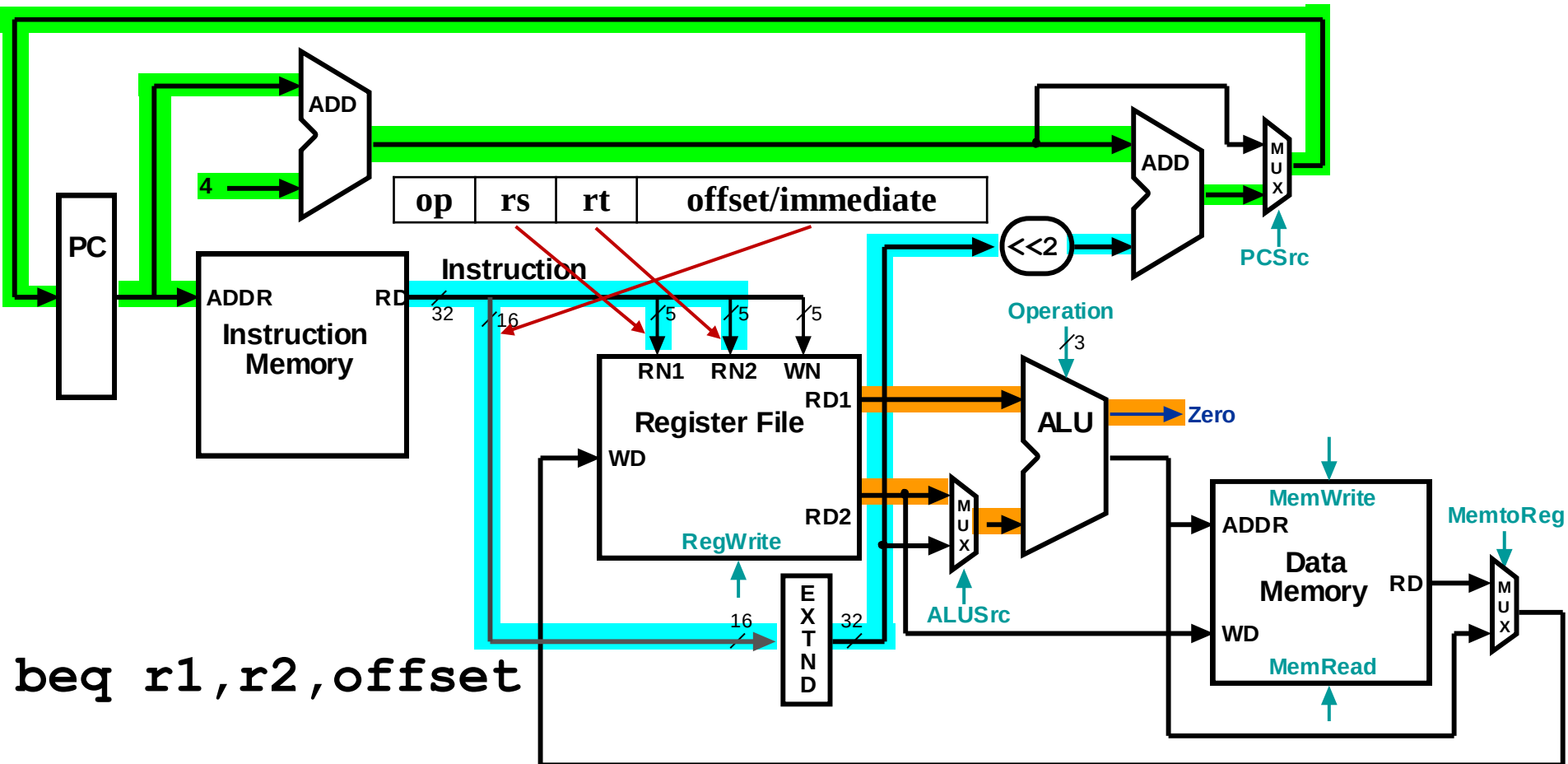


Datapath Executing sw





Datapath Executing beq

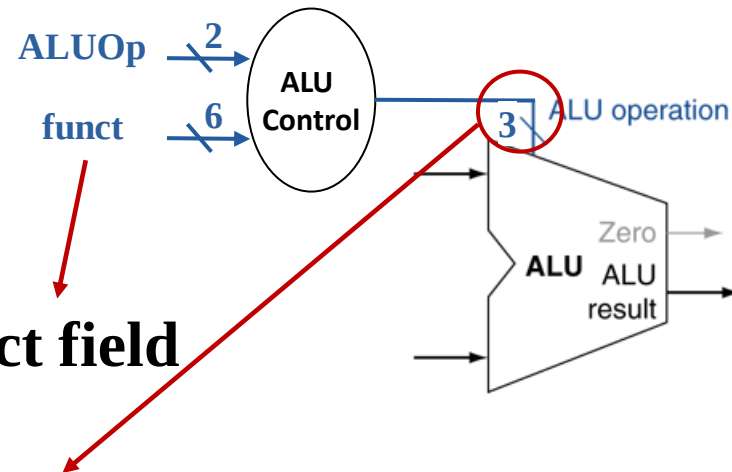




ALU Control (1)

■ ALU used for

- ◆ Load/Store: F = add
- ◆ Branch: F = subtract
- ◆ R-type: F depends on funct field



ALU control	Function
000	AND
001	OR
010	add
110	subtract
111	set-on-less-than





ALU Control (2)

- Assume 2-bit ALUOp derived from opcode
 - ◆ Combinational logic derives ALU control

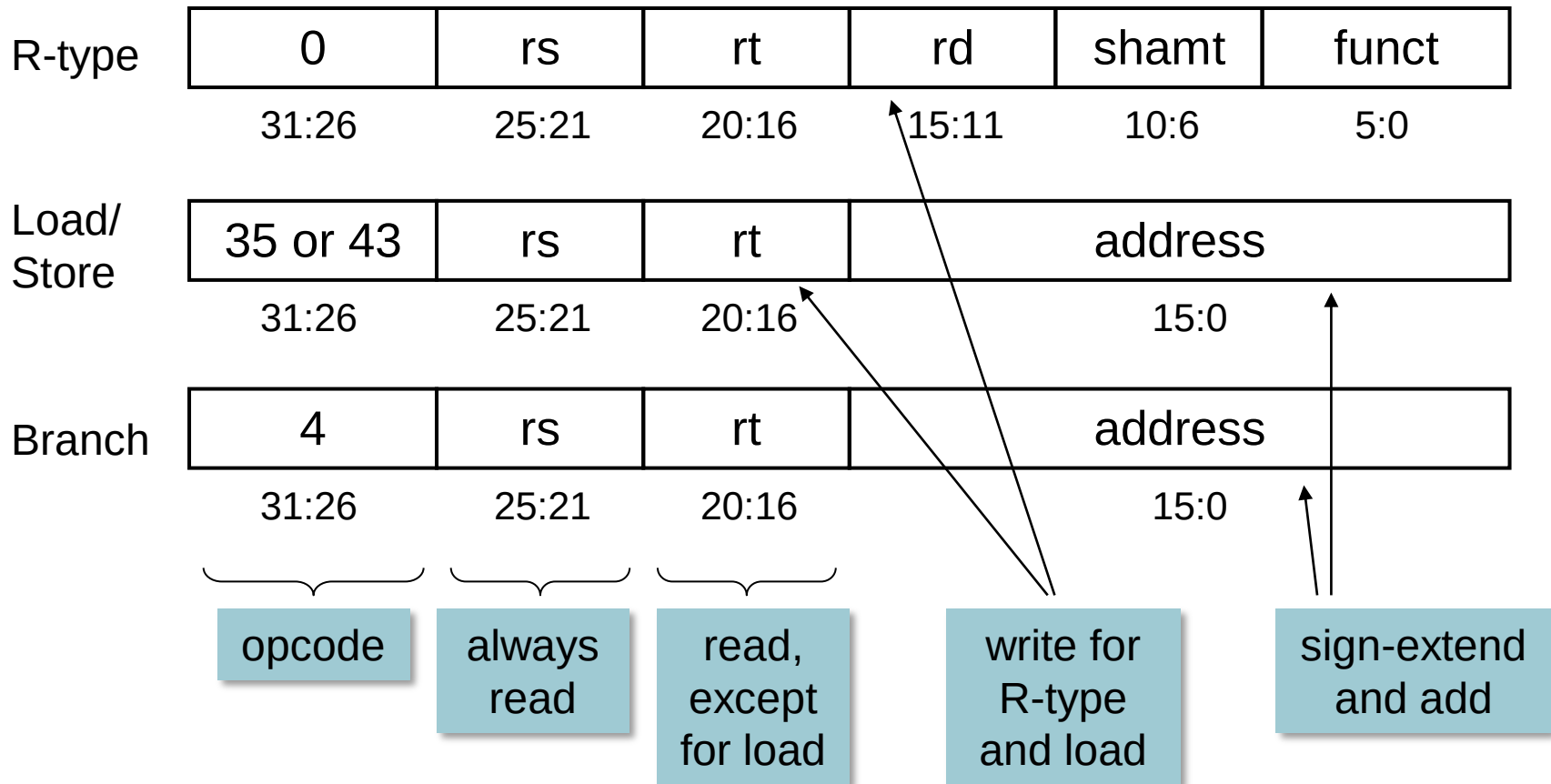
opcode	ALUOp	Operation	funct	ALU function	ALU control
lw	00	load word	XXXXXX	add	0010
sw	00	store word	XXXXXX	add	0010
beq	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
		subtract	100010	subtract	0110
		AND	100100	AND	0000
		OR	100101	OR	0001
		set-on-less-than	101010	set-on-less-than	0111





The Main Control Unit

■ Control signals derived from instruction





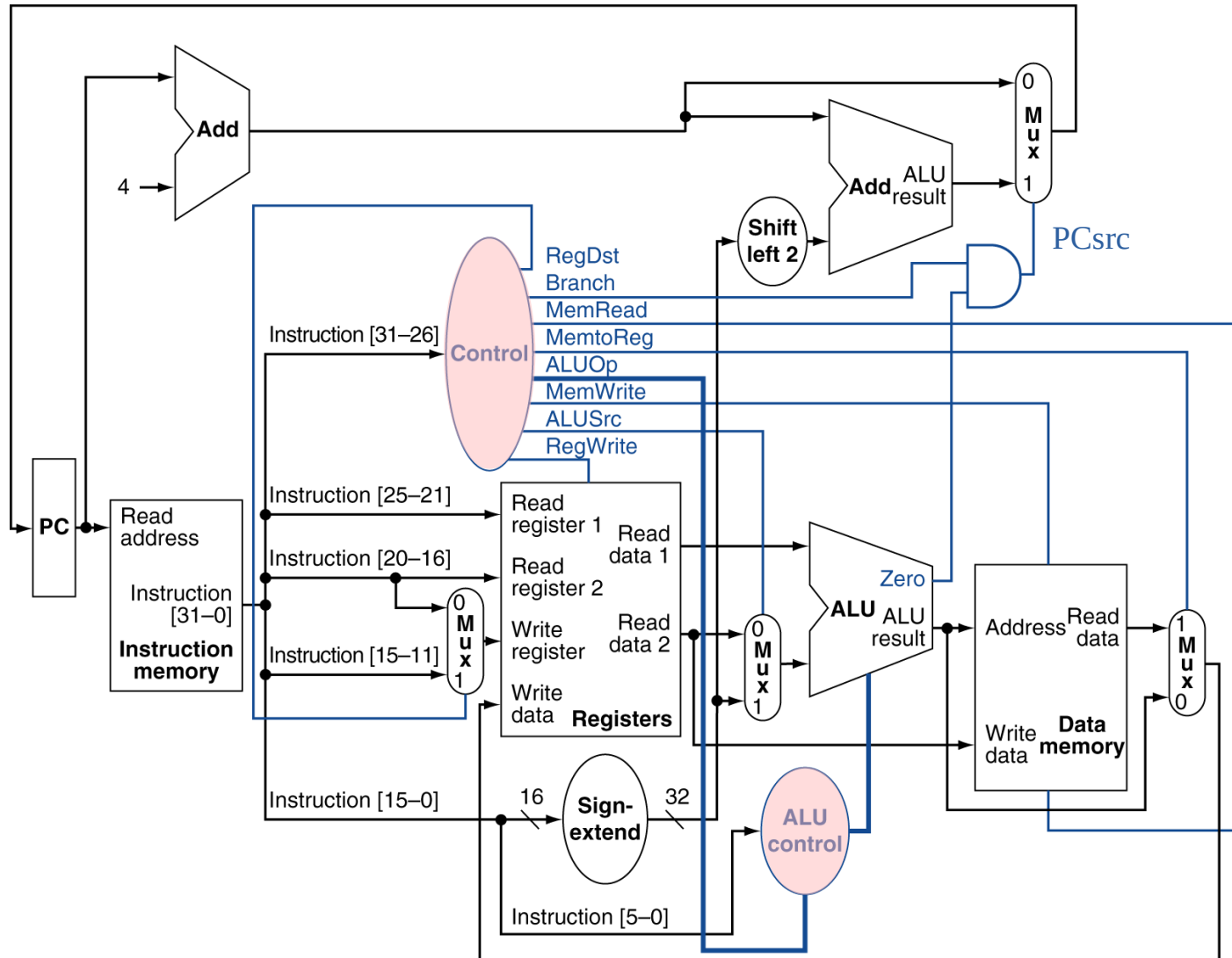
Main Control Truth Table

		0	35	43	4
输入或输出	信号名	R 型	lw	sw	beq
输入	Op5	0	1	1	0
	Op4	0	0	0	0
	Op3	0	0	1	0
	Op2	0	0	0	1
	Op1	0	1	1	0
	Op0	0	1	1	0
输出	RegDst	1	0	X	X
	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

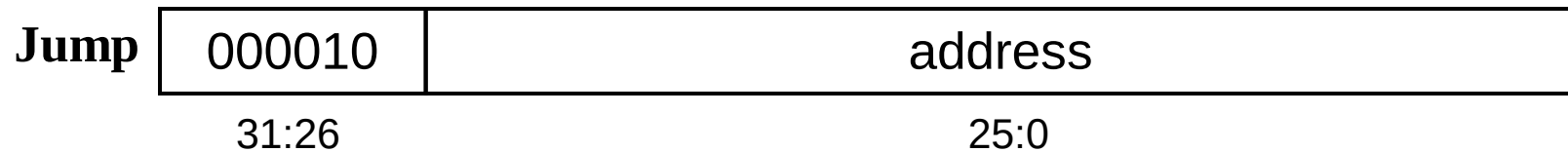




Datapath With Control



Implementing Jumps

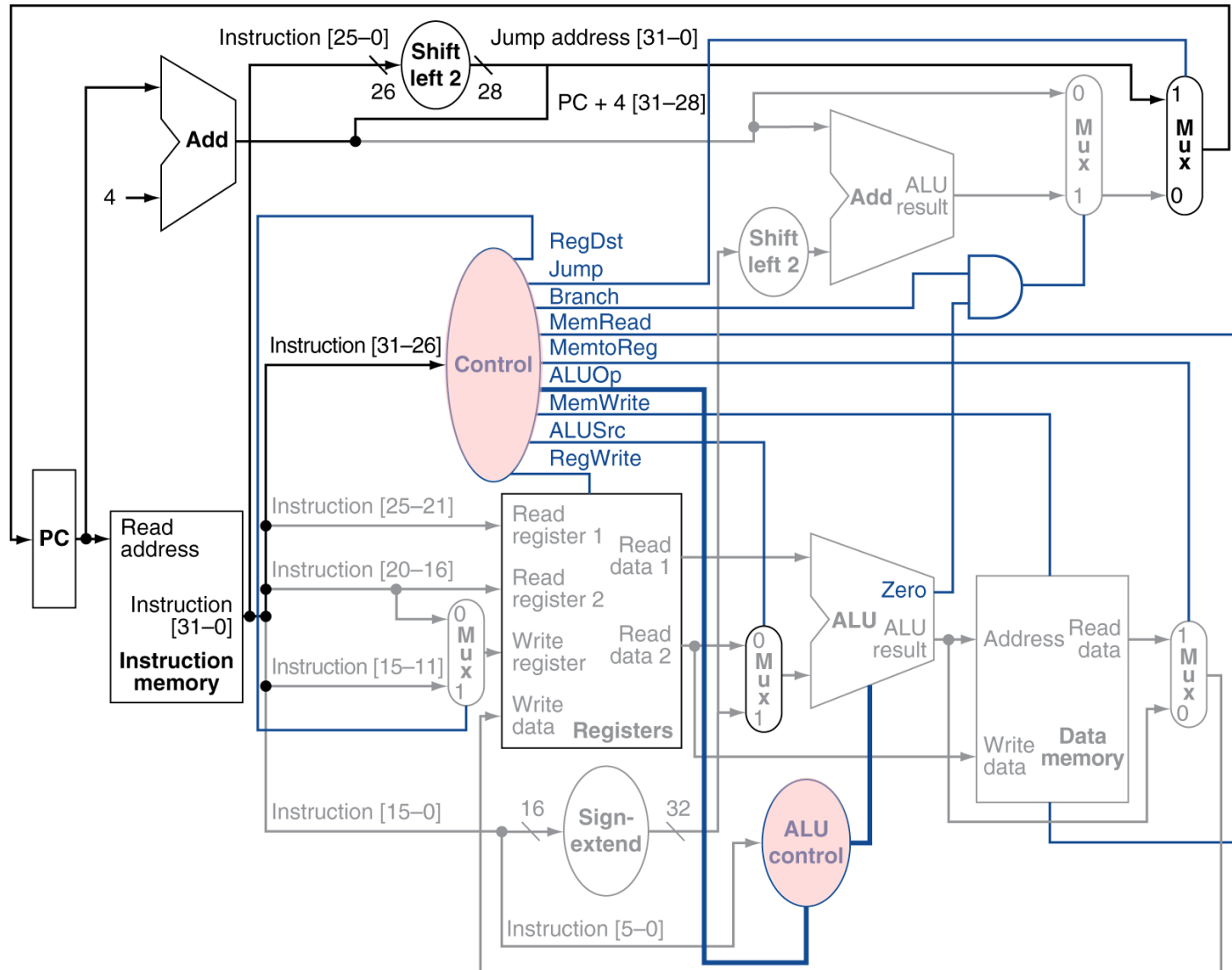


- Jump uses **word address**
- Update PC with concatenation of
 - ◆ Top 4 bits of old PC
 - ◆ 26-bit jump address
 - ◆ 00
- Need an extra control signal decoded from opcode





Datapath With Jumps Added





性能问题

- 最长的延迟决定时钟周期
 - ◆ 关键路径：load指令
 - ◆ 指令存储器→寄存器堆→ALU→数据存储器→寄存器堆
- 对于各指令的执行时间差异较大的处理器，采用单周期的设计方案效率太低
- 违背了“加速大概率事件”的设计原则
- 可采用流水线技术改进性能





5.6 流水线CPU





并行处理技术

- 早期冯·诺伊曼体系结构计算机，采用串行处理
 - ◆ 计算机的访存、算术逻辑运算等操作只能串行完成
 - ◆ 任一时刻只能进行一个操作
- 并行性（**Parallelism**）概念
 - ◆ 问题中具有可以同时进行运算或操作的特性
 - ◆ 例：在相同时延的条件下，用 n 位运算器进行 n 位并行运算速度几乎是1位运算器进行 n 位串行运算的 n 倍（狭义）
- 广义并行含义
 - ◆ 只要在同一时刻（同时性）或在同一时间间隔内（并发性）完成两种或两种以上性质相同或不同的工作，它们在时间上相互重叠，体现出了并行性





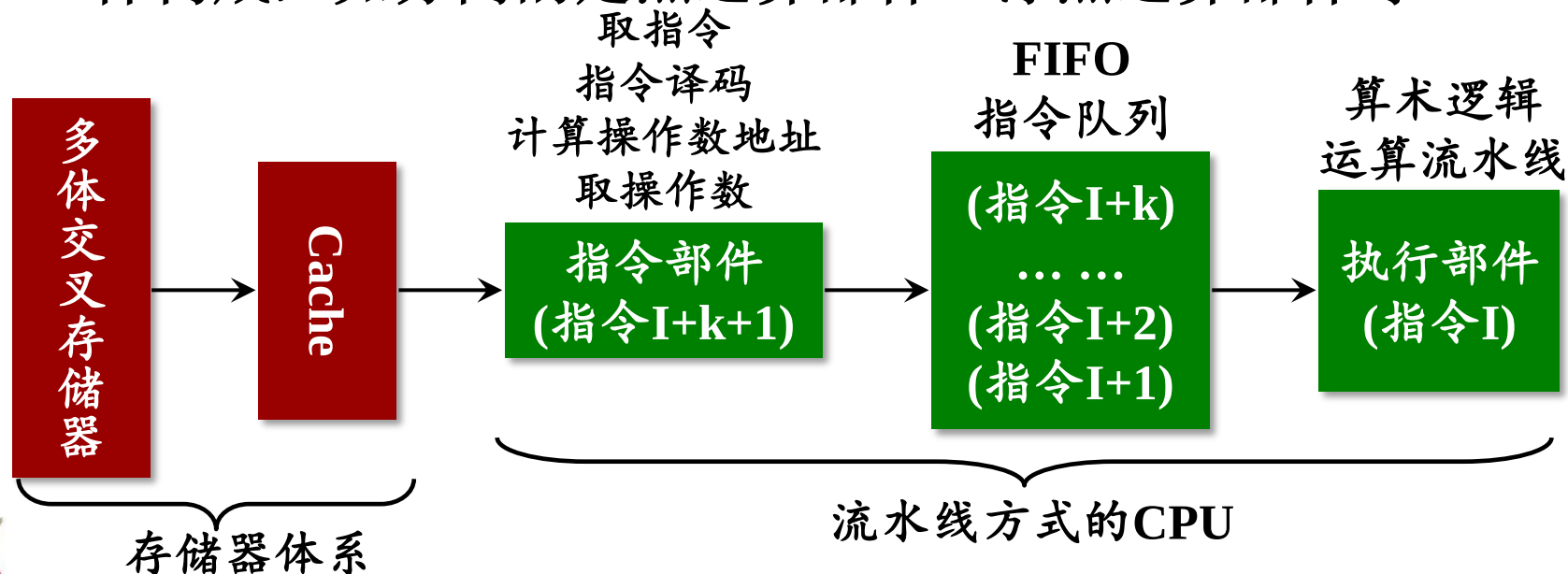
并行处理技术的三种形式

- 时间并行（重叠）：让多个处理过程在时间上相互错开，轮流重叠地使用同一套硬件设备的各个部件，加快硬件周转而赢得速度。实现方式就是采用流水处理部件
- 空间并行（资源重复）：以数量取胜
 - ◆ 能真正的体现同时性
 - ◆ LSI和VLSI为其提供了技术保证
 - ◆ 如：多处理器系统和多计算机系统
- 时间+空间并行
 - ◆ Pentium中采用超标量流水线技术



流水线计算机的系统组成

- 存储器体系：主存采用多体交叉存储器； Cache
- 流水线（pipelining）方式CPU
 - ◆ 指令部件：取指令、译码、计算操作数地址、取操作数
 - ◆ 指令队列：FIFO，存放译码后指令和取来的操作数
 - ◆ 执行部件：可以有多个采用流水线方式构成的算术逻辑部件构成，如分离的定点运算部件、浮点运算部件等





指令处理的分解

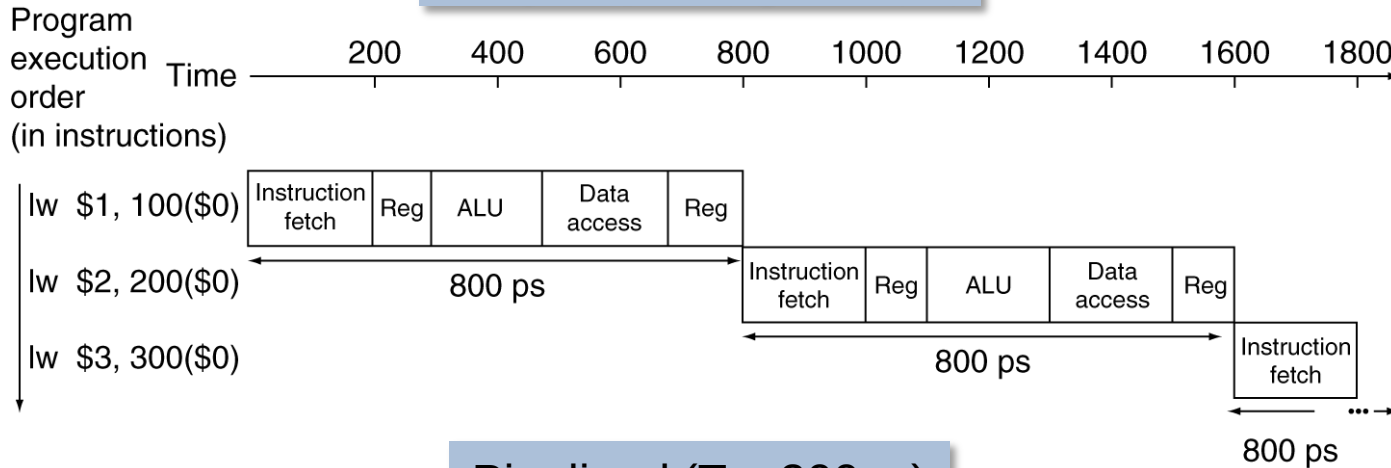
- 把指令的处理过程分解为多个处理步骤，让多条指令能在各处理步骤**重叠**执行，从而实现以流水线方式执行指令，达到加快指令总体执行时间的目的
- **IF (Instruction Fetch取指)**
 - ◆ 从存储器取指令
- **ID (Instruction Decode指令译码)**
 - ◆ 指令译码、读寄存器
- **EX (Execution执行)**
 - ◆ 执行指令或计算地址
- **MEM (Memory)**
 - ◆ 访问存储器
- **WB (Write Back写回)**
 - ◆ 结果写回寄存器



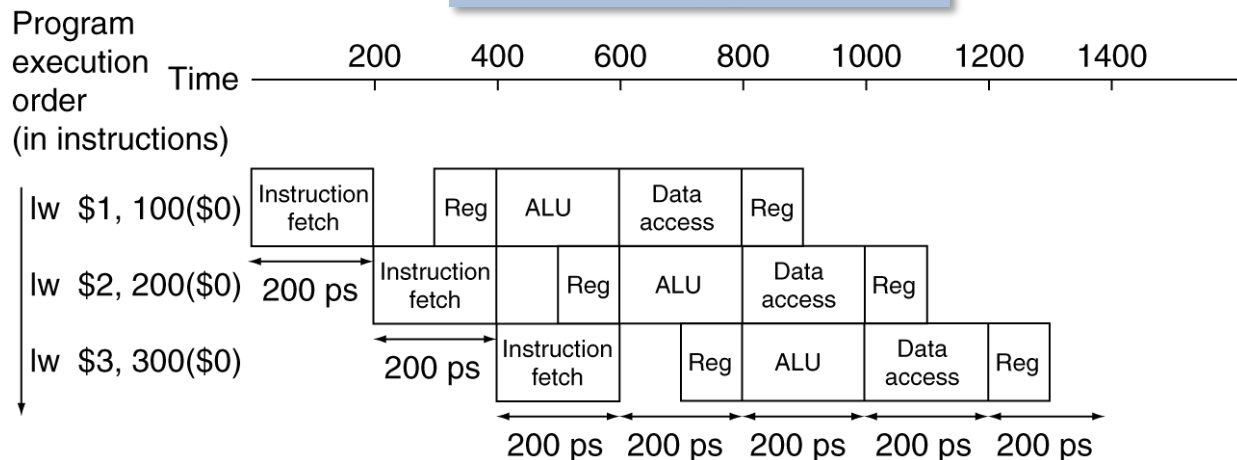


单周期指令 vs. 流水线

Single-cycle ($T_c = 800\text{ps}$)

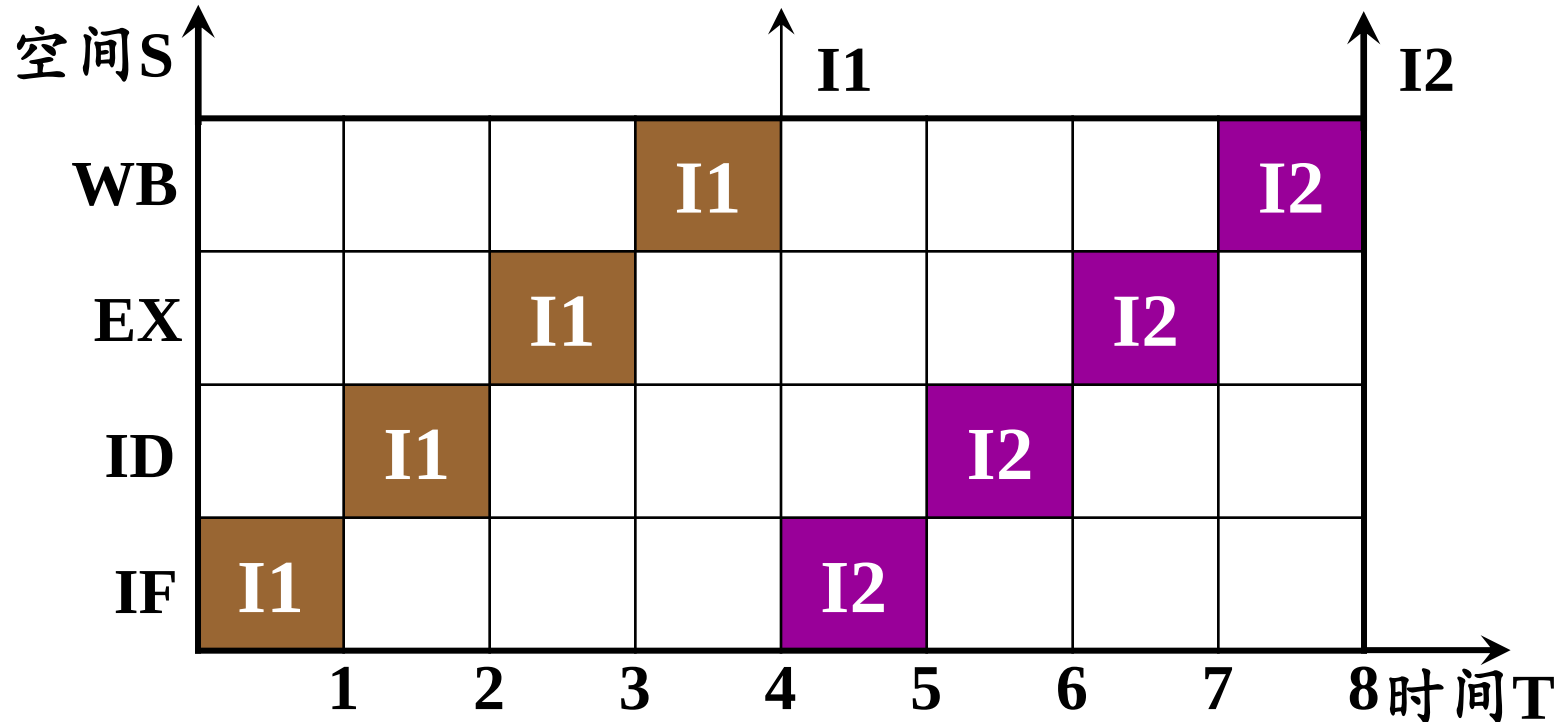


Pipelined ($T_c = 200\text{ps}$)





非流水线时空图

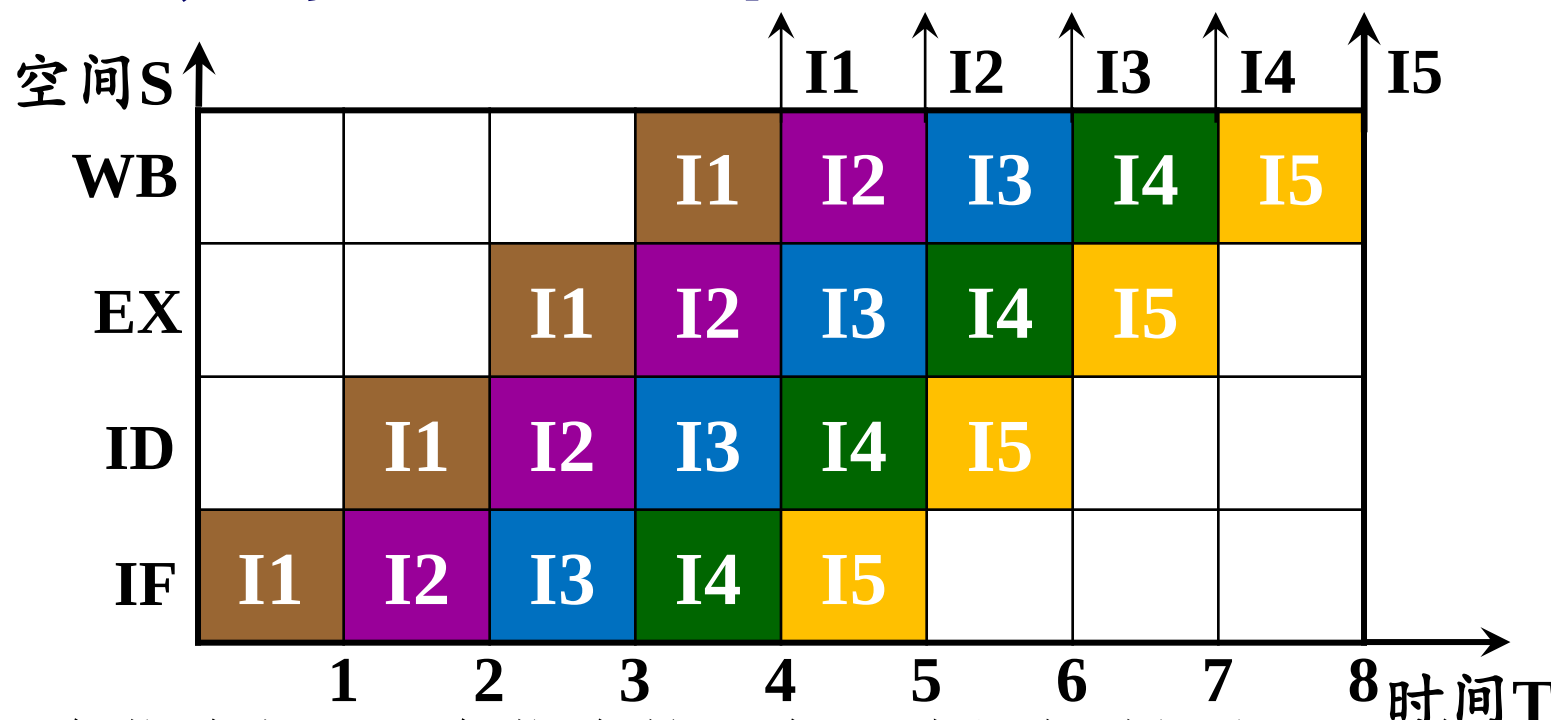


- 上一条指令的四个子过程全部被执行完毕后才能开始下一条指令
- 每隔4个机器时钟周期才有一个输出结果





标量流水线时空图

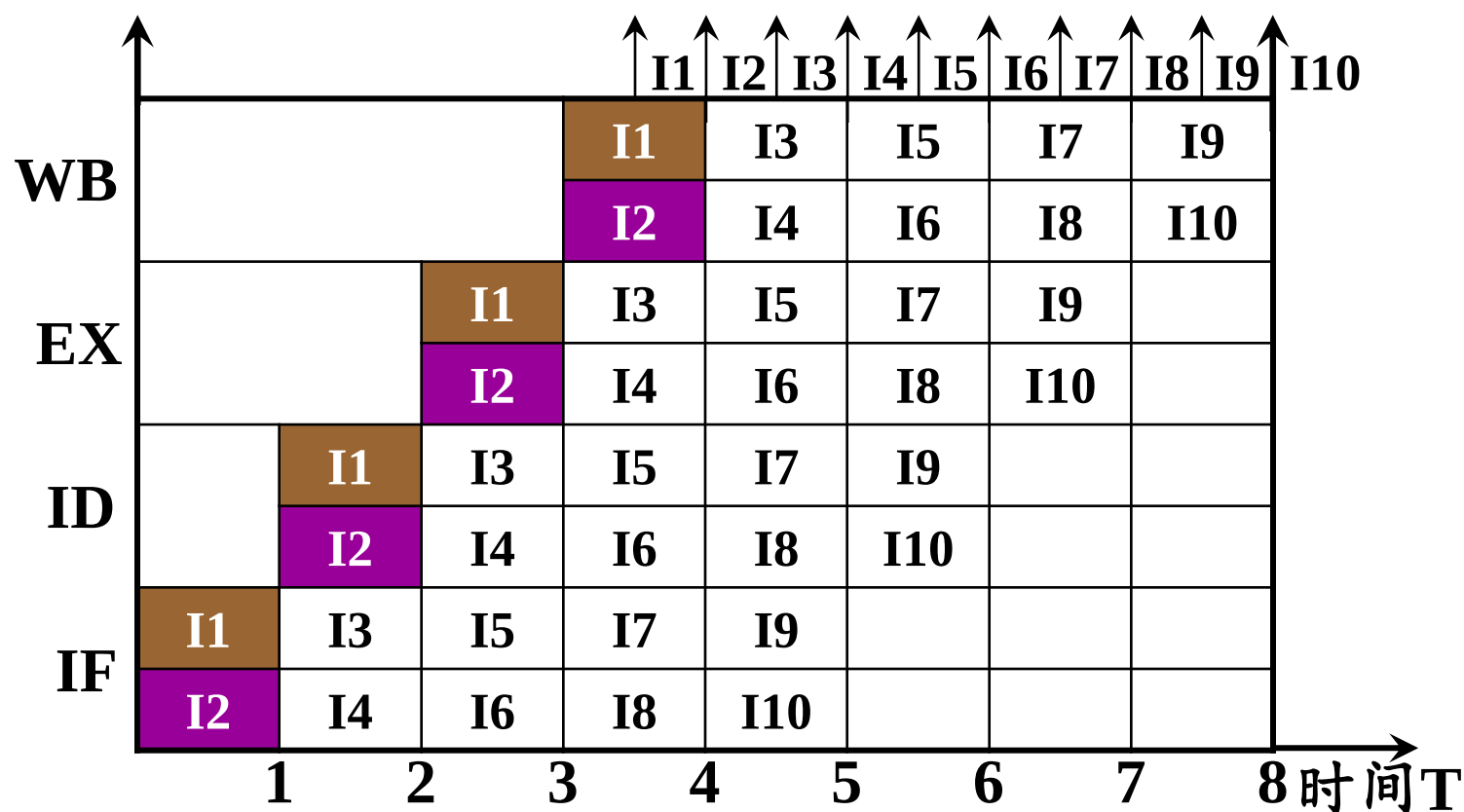


- 上一条指令与下一条指令的四个子过程在时间上可以输出重叠执行
- 每一个时钟周期就可以输出一个结果
- 对比
 - ◆ 流水计算机在8个单位时间中执行了5条指令
 - ◆ 非流水线计算机在8个单位时间中仅执行了2条指令





超标量流水线时空图



◆ 超标量流水线计算机在8个单位时间中执行了10条指令





流水线分类

■ 指令流水线

- ◆ 指令步骤的并行

■ 算术流水线

- ◆ 运算操作步骤的并行
- ◆ 如流水线加法器、流水线乘法器、流水线除法等

■ 处理机流水线

- ◆ 程序步骤的并行，也称为宏流水线
- ◆ 多台级联的处理机构成流水线的各过程段，每台处理机负责某一特定的任务
- ◆ 处理机流水线应用在多机系统中





流水技术原理

- 把一个任务分解为几个有联系的子任务。每个子任务由一个专门的功能部件实现
- 在流水线中的每个功能部件之后都要有一个缓冲寄存器，或称为锁存器
- 流水线中各段的时间应该**尽量相等**，否则将会引起“堵塞”和“断流”的现象
- 在流水线中必须是连续的任务，只有不断的提供任务才能充分发挥流水线的效率
- 流水线需要有装入时间和排空时间，只有当流水线完全充满时，才能充分发挥效率





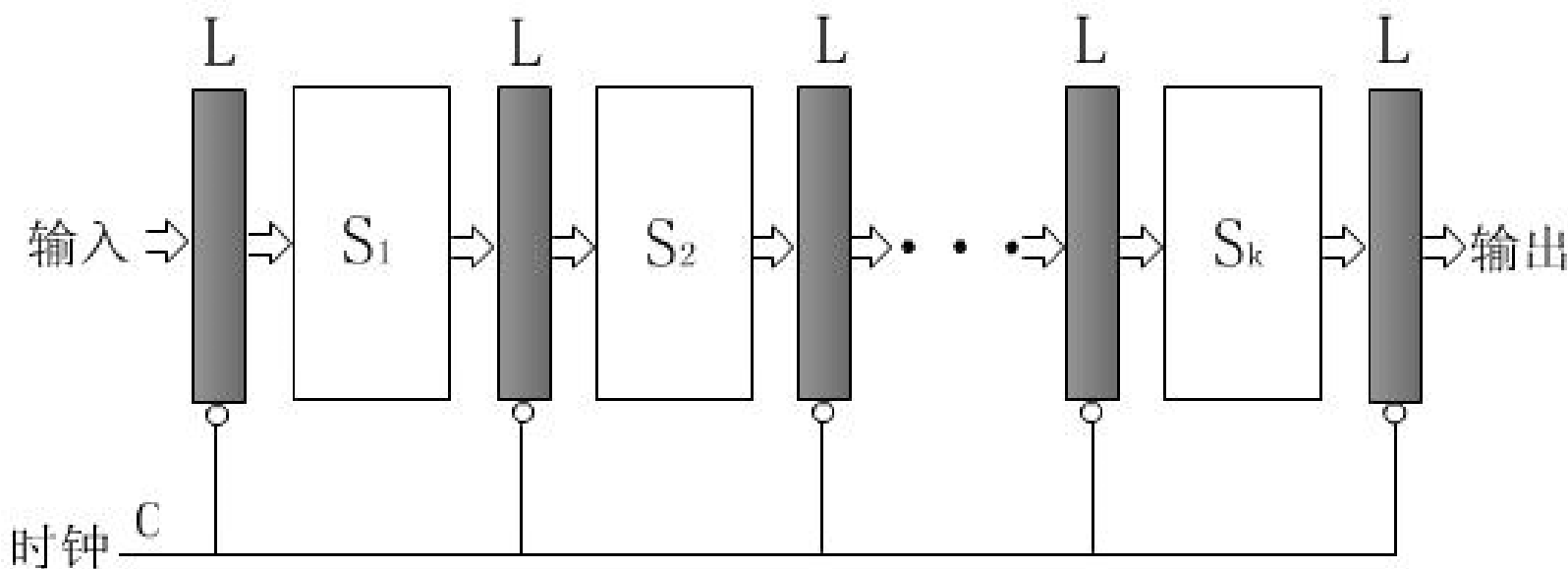
流水硬件基本结构

设阶段 S_i 所需的时间为 τ_i , 缓冲寄存器的延时为 τ_l ,
线性流水线的时钟周期定义为

$$\tau = \max \{ \tau_i \} + \tau_l = \tau_m + \tau_l$$

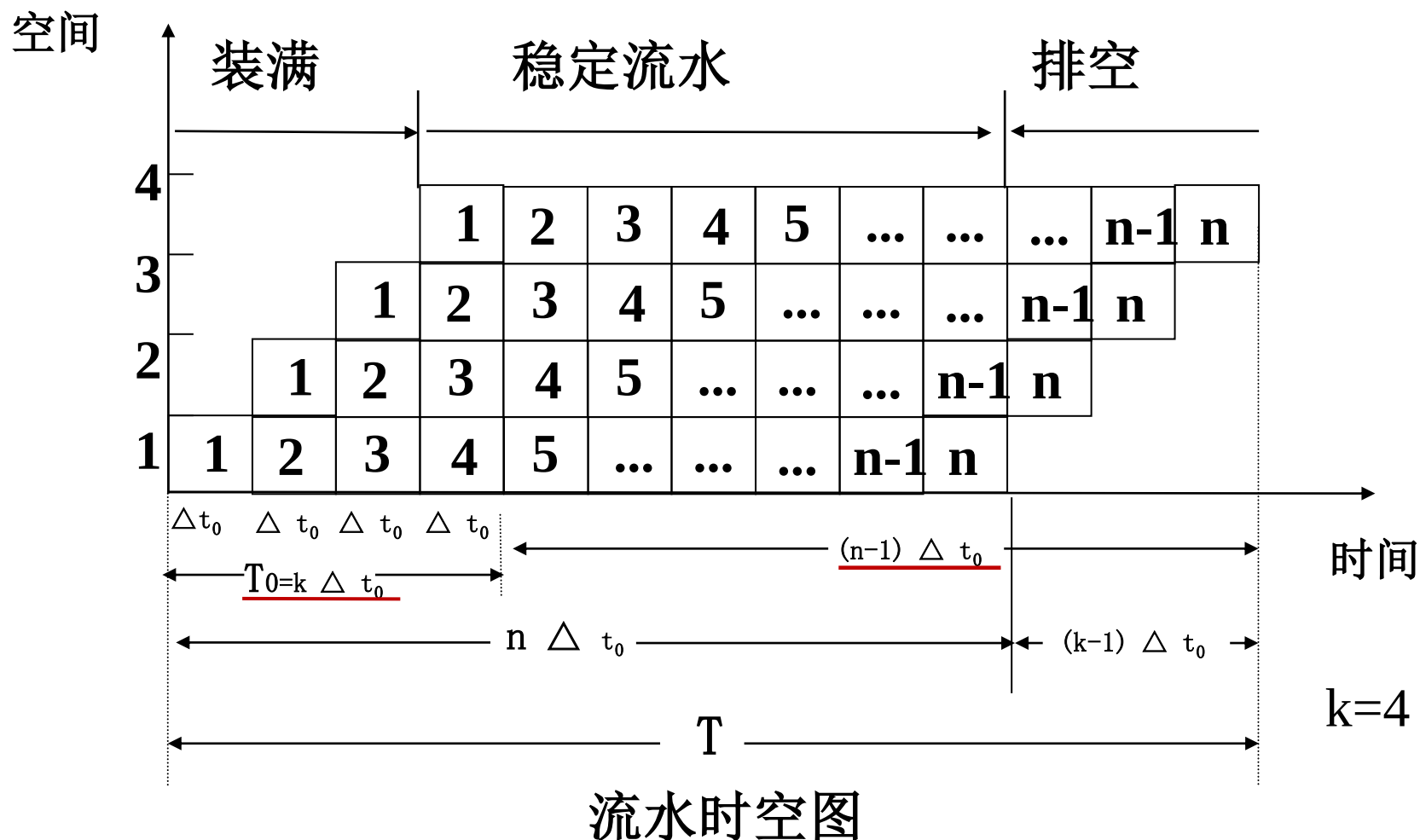
流水线处理的频率为

$$f = 1 / \tau$$





线性流水线的三个阶段





流水线的性能指标（1）

- 流水线的加速比（speedup ratio）：衡量在不使用流水线和使用流水线两种情况下完成n个任务所用的时间比：

$$S = T_0 / T_k$$

假设k级流水线的时钟周期为 τ ，则

不使用流水线顺序执行n个任务所需时间：

$$T_0 = n \cdot k \cdot \tau$$

使用k级流水线完成n个连续任务所需时间：

$$T_k = k \cdot \tau + (n-1) \tau = (k+n-1) \cdot \tau$$

故： $S = n \cdot k \cdot \tau / (k+n-1) \tau = n \cdot k / (k+n-1)$

在 $n \gg k$ 的情况下，最大加速比为k





流水线的性能指标（2）

■ 吞吐率

$$TP = n / T_k$$

其中 n 为指令条数， T_k 为执行 n 条指令所用时间

$$TP = \frac{n}{T_k} = \frac{n}{(k+n-1)\tau}$$

■ 最大吞吐率

$$TP_{\max} = \lim_{n \rightarrow \infty} \frac{n}{(k+n-1)\tau} = \frac{1}{\tau}$$

补讲 第2章II





流水线的特点

- 流水线操作并不能加快任何一条指令的执行过程，只能加快连续一串指令的执行过程
- 一种实现多条指令重叠执行的技术
- 潜在加速比= 流水线级数
- 流水线的速率受限于最慢的流水段
- 流水段的执行时间若不均衡，则加速比就会降低
- 开始填充流水线和最后排空流水线的时间降低了加速比
- 可能出现在下一个时钟周期中下一条指令不能执行的情况，称之为**冒险**（hazard），也称为相关





三种流水线冒险

- 结构冒险（**structural hazard**）/资源相关：指多条指令进入流水线后在同一时钟周期内争用同一个资源所发生的冲突
- 数据冒险（**data hazard**）/数据相关：一条指令的执行依赖于更早的一条还在流水线中指令的执行结果
- 控制冒险（**control hazard**）/控制相关：由条件转移指令（或称分支指令）引起。即：因为取到的指令并不是所需要的从而导致指令不能在预定的时钟周期内执行

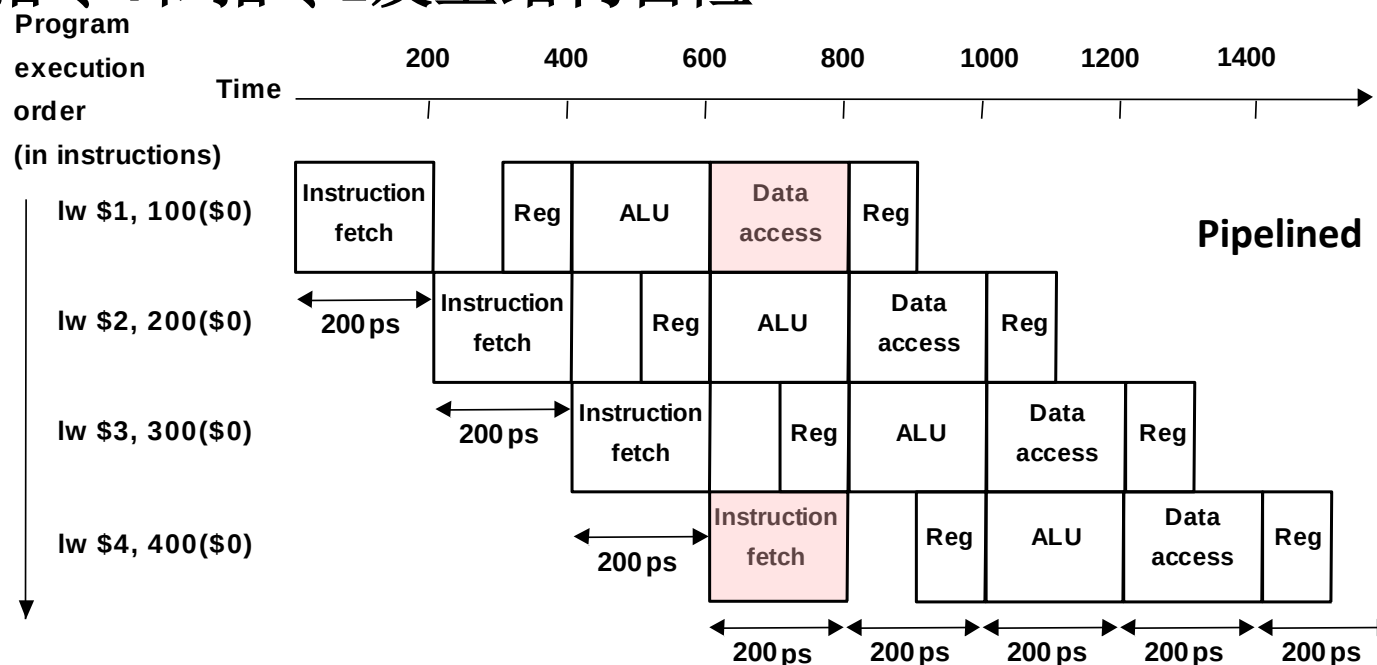




结构冒险

- 因缺乏足够硬件支持而导致流水线中的指令不能在同一个时钟周期内执行。或:流水线中两条指令同时试图使用同一资源

例: 指令4和指令1发生结构冒险



- 消除结构冒险的方法

- ◆ 暂停一个时钟

- ◆ 增设一个存储器，将指令和数据放在不同的存储器中

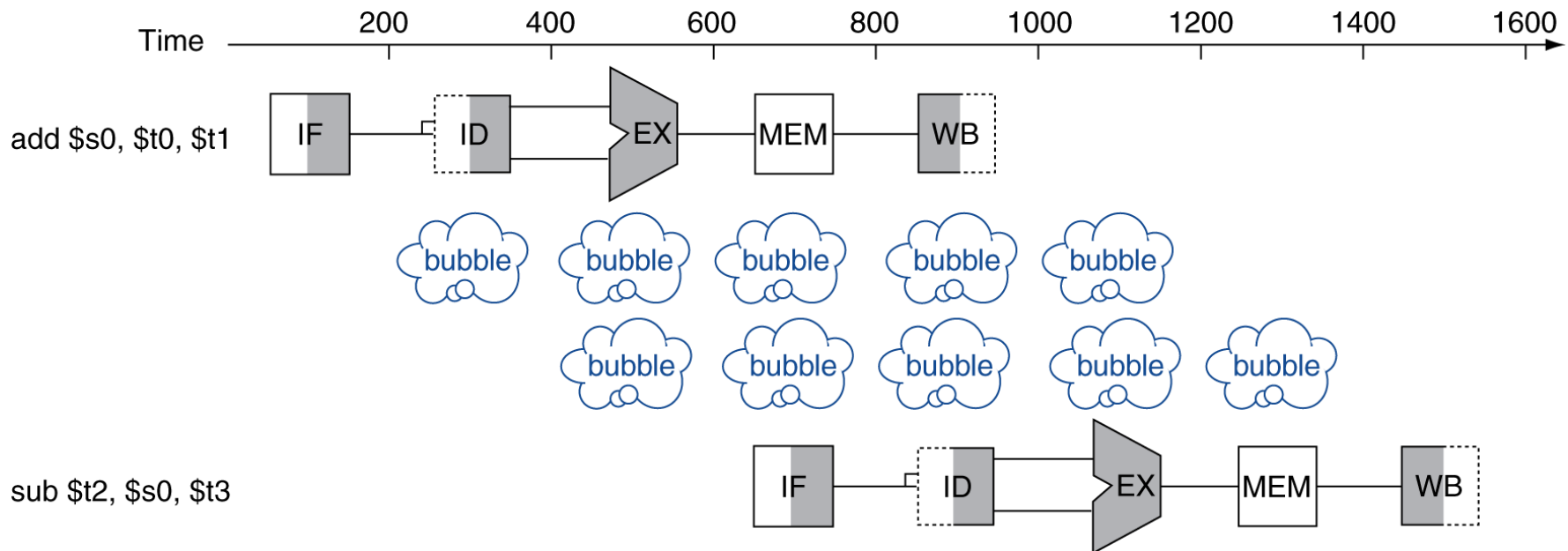




数据冒险

- 一条指令的执行依赖于更早的一条还在流水线中指令的执行结果

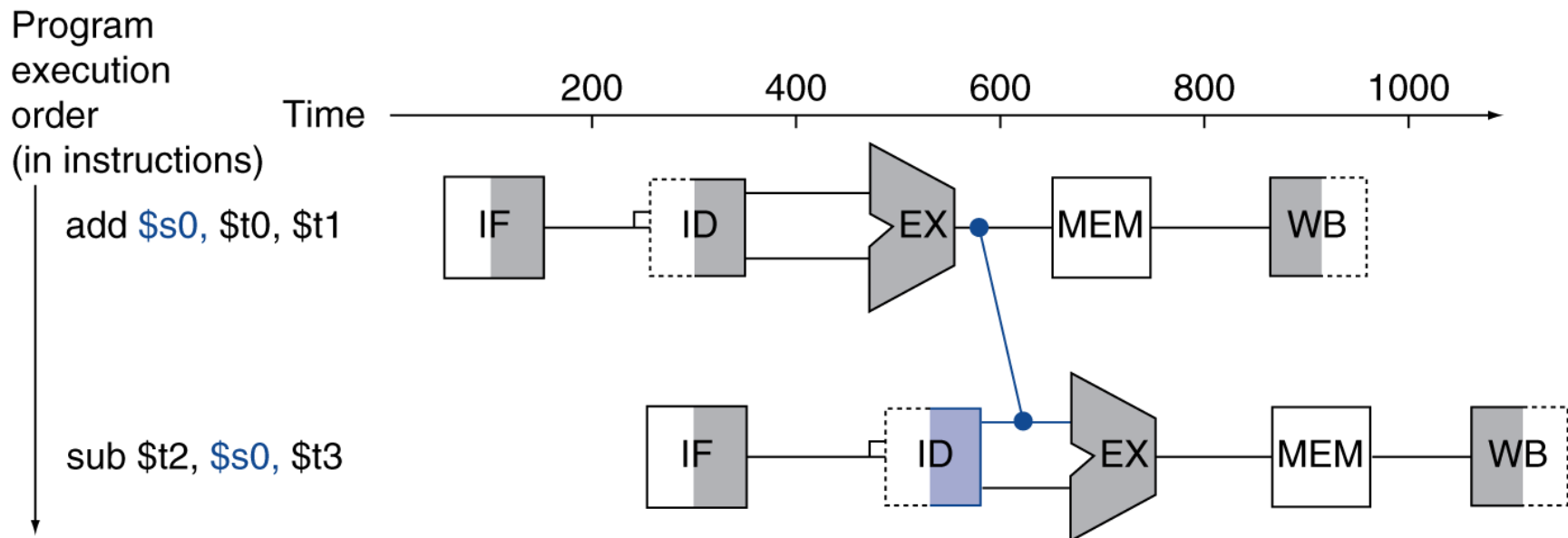
```
add    $s0, $t0, $t1
sub     $t2, $s0, $t3
```





前推/旁路(Forwarding 或 Bypassing)

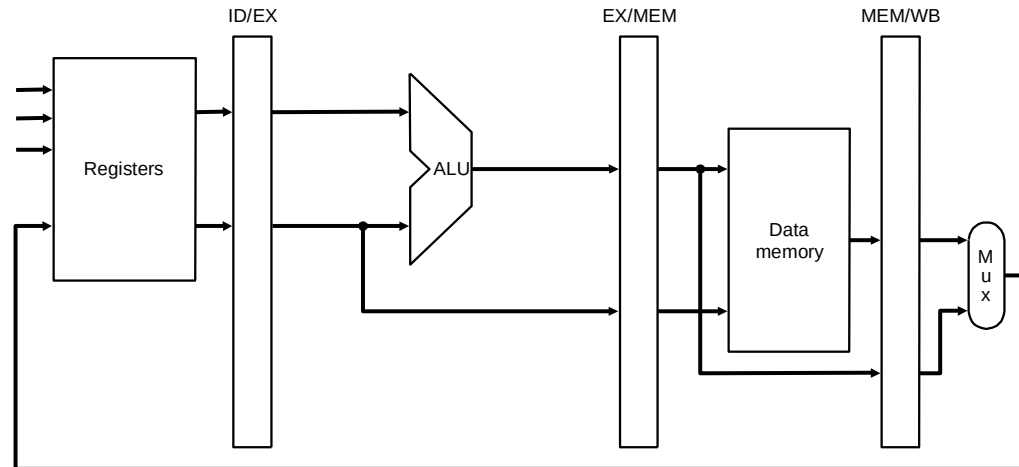
- 直接使用刚计算出的结果
 - ◆ 无需等待该结果写入寄存器
 - ◆ 数据通路中需要额外的连接





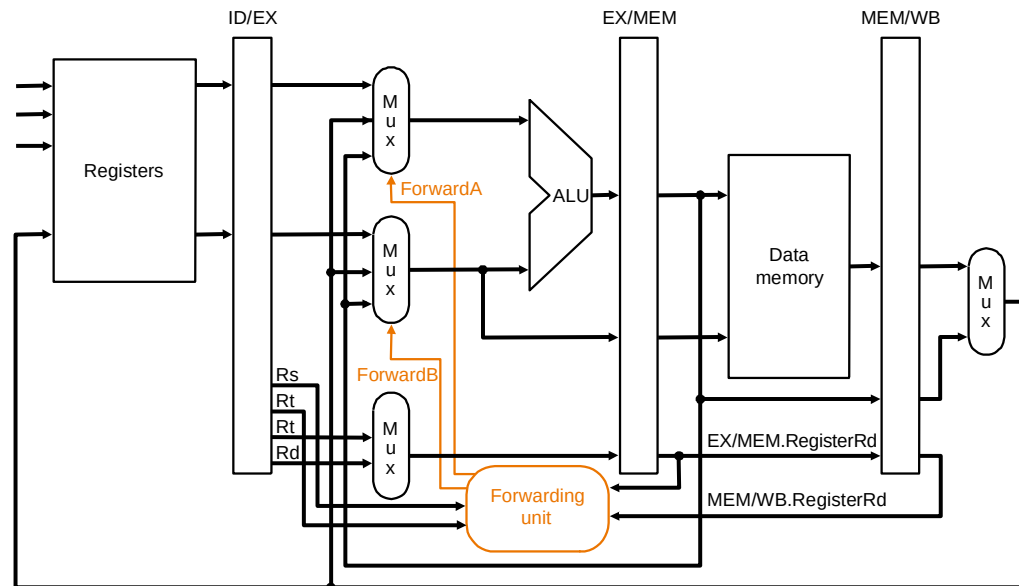
Forwarding的实现

未使用Forwarding



a. No forwarding

增加Forwarding



b. With forwarding





控制冒险

- 控制冒险是由分支（条件转移）指令引起的。因为取下一条指令依赖于当前分支指令的结果，而流水线不能保证总是能取到正确的指令，这将导致流水线阻塞

```
if (i != j)
    f = g + h;
else
    f = g - h;
```



```
beq $s3, $s4, else
add $s0, $s1, $s2
j next
else: sub $s0, $s1, $s2
next:
```

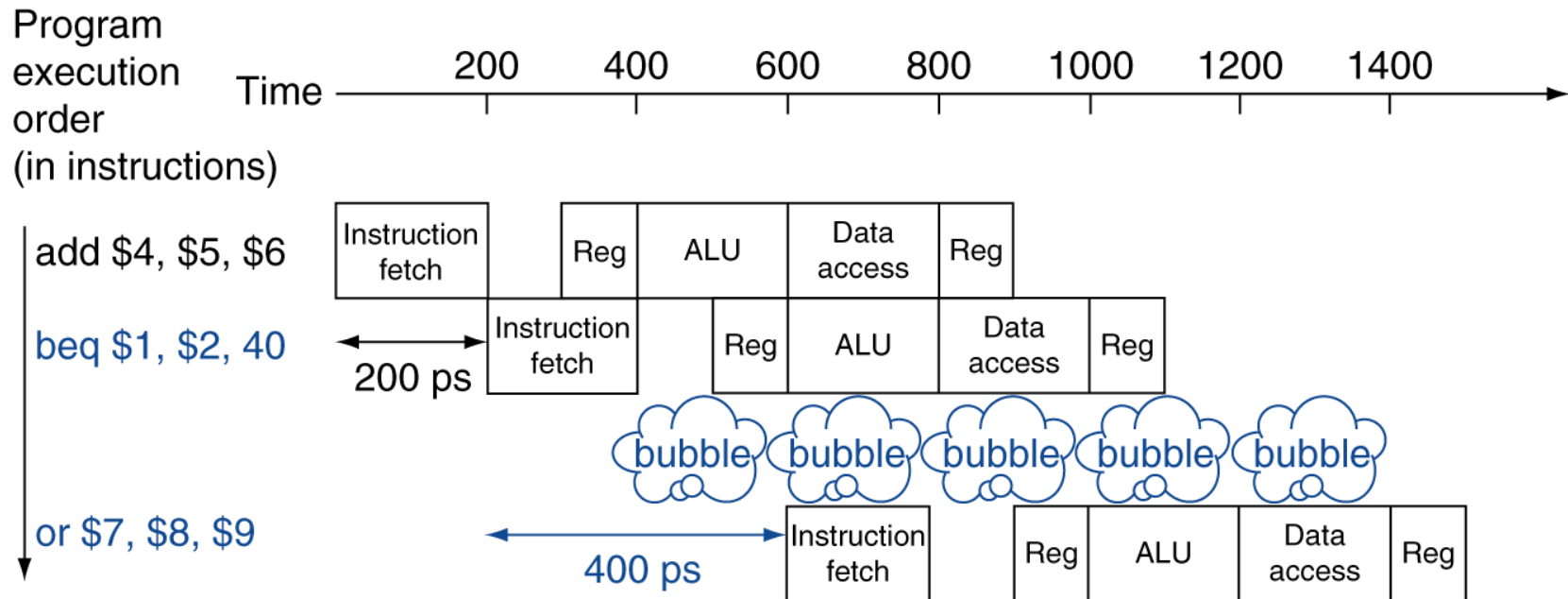
设C代码中的f、g、h、i和j变量依次对应与\$s0 ~ \$s4寄存器





分支指令的阻塞

- 等待直到分支指令结果出来后才取下一条指令
- 下图为MIPS流水线
 - ◆ 增加专门硬件，在指令译码ID（第二）阶段完成寄存器比较和目标地址计算





消除控制冒险方法 – 分支预测

分支预测（Branch Prediction）：猜测分支方向并根据猜测开始取指的技术

■ 静态分支预测（static branch prediction）

- ◆ 简单静态预测
- ◆ 复杂静态预测

■ 动态分支预测（dynamic branch prediction）

- ◆ 观察上一次执行该分支指令时分支是否发生，如果上次执行时分支发生，就从上次发生分支的地方开始取新指令





静态分支预测

■ 简单静态预测

- ◆ 预测分支不发生 (**always not taken**)
- ◆ 预测分支发生 (**always taken**)

■ 复杂静态预测

- ◆ 编译器在编译时分析程序完成对指令序列的重排，从而消除控制冒险
 - 从分支指令进入流水线到得到结果期间（称之为分支延迟时间槽 **branch delay slot**），有后续若干指令已进入流水线，如果分支发生，则那些后续已进入流水线并译码的指令需要废弃，导致流水线断流
 - 解决方法：在延迟槽中安排如下指令：分支指令之前的指令或分支指令目标处的指令
 - 这种重排指令序列的处理由编译程序来完成，且指令执行顺序的变动不能影响到程序的正确性。

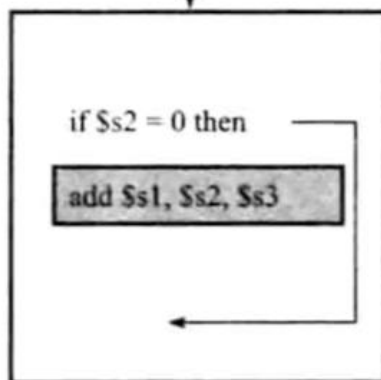


分支延迟时间槽的指令重排

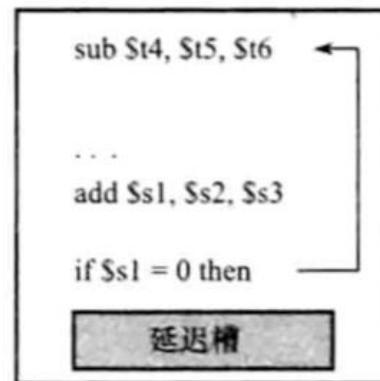
重排前的
代码



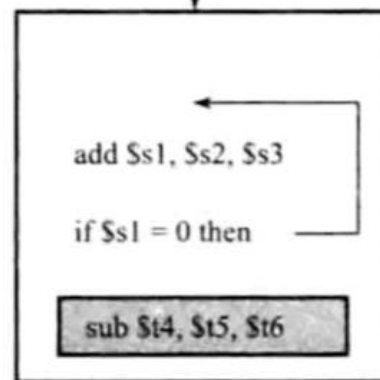
变成



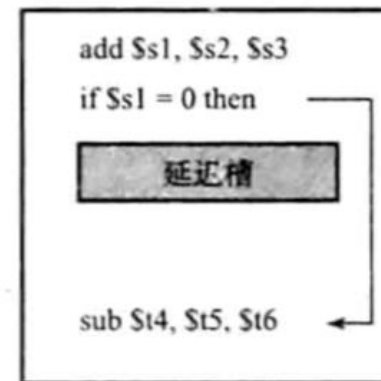
a) 从前面调度



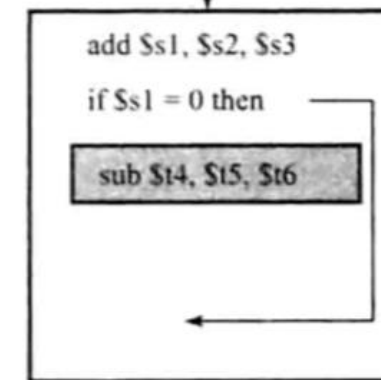
变成



b) 从目标处调度



变成



c) 从不发生转移处调度

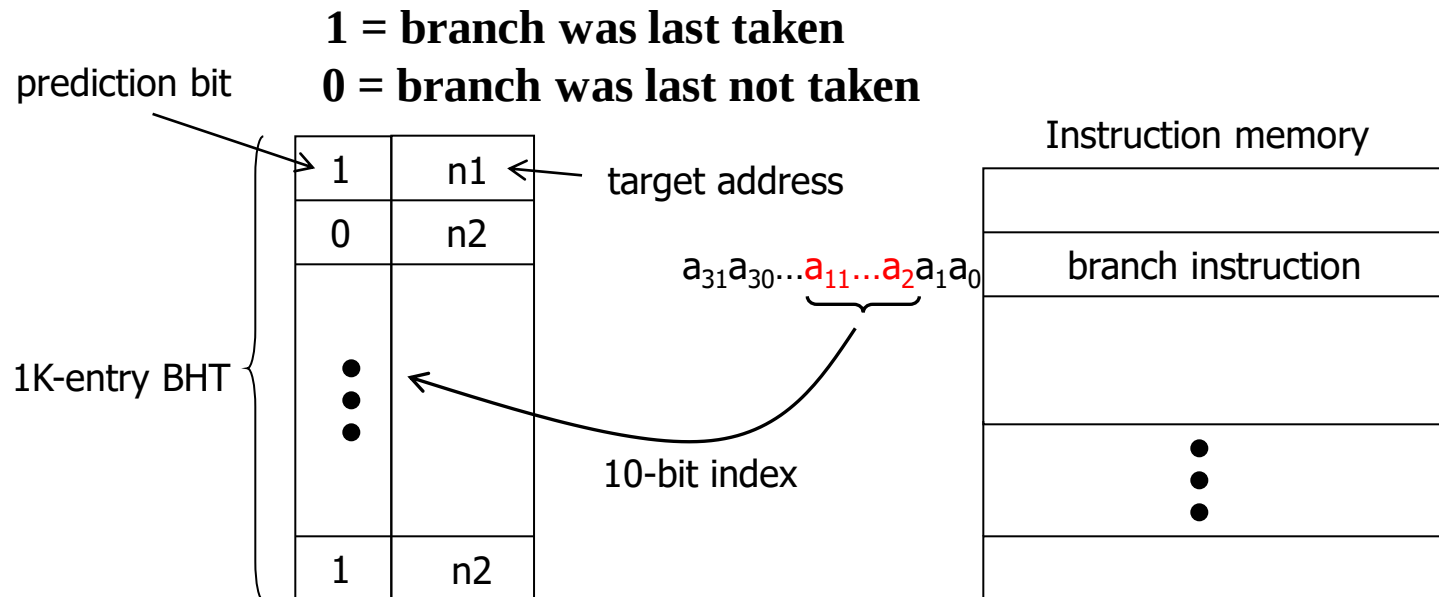
重排后
的代码

指令执行顺序的重排不能影响到程序的正确性



动态分支预测

- 根据一条分支指令过去的行为来预测该指令将来的行为，需用硬件方法实现
- 处理器维护一张**分支历史表BHT**（**分支预测缓存BPB**），该表使用分支指令地址的低位部分（如10位）进行索引，每一表项的预测位记录了最近一次是否发生过分支，以及记录了目标地址等内容。如果预测错误，将预测位取反





例4

流水线中有三类数据相关冲突：写后读相关（**RAW**）；读后写相关（**WAR**），也称反相关（**antidependence**）；写后写相关（**WAW**），也称输出相关。判断以下三组指令各存在哪种类型的数据相关：

(1) I1: ADD R1,R2,R3 ; (R2) + (R3) → R1

写后读

I2: SUB R4,R1,R5 ; (R1) - (R5) → R4

(2) I3: STO M(x),R3 ; (R3) → M(x), M(x)是存储器单元

读后写

I4: ADD R3,R4,R5 ; (R4) + (R5) → R3

(3) I5: MUL R3,R1,R2 ; (R1) × (R2) → R3

写后写

I6: ADD R3,R4,R5 ; (R4) + (R5) → R3





解

第(1)组指令中，I1指令运算结果应先写入R1，然后在I2指令中读出R1内容。由于I2指令进入流水线，变成I2指令在I1指令写入R1前就读出R1内容，发生RAW相关。

第(2)组指令中，I3指令读出R3内容前I4指令不能将运算结果写入R3，发生WAR相关。在简单顺序流水线上不会发生WAR冲突，而乱序执行时，若I4先于I3执行情况就不一样了。可对寄存器重命名消除反相关

第(3)组指令中，根据指令顺序，R3的最后结果来自于ADD指令，如果乱序执行时，I6先于I5执行，则出现I5指令试图向后一条指令I6已经写过R3中再次写入，导致R3内容错误，发生WAW相关。可对寄存器重命名，或通过丢弃ADD指令结果而不写回消除反相关





动手做

1、下列不会引起指令流水阻塞的是 ()

A: 数据旁路

B: 数据相关

C: 条件转移

D: 资源冲突

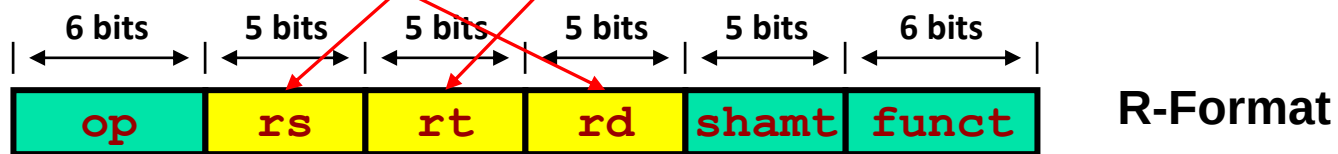




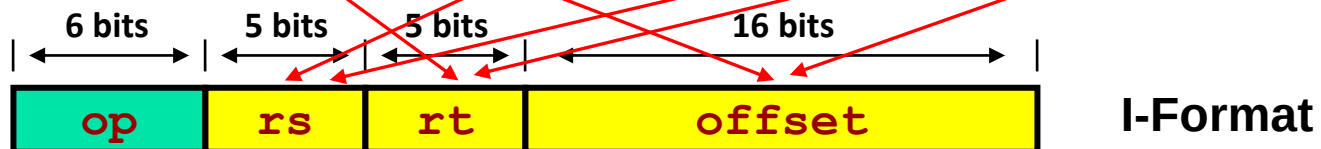
实例 MIPS指令格式

- 算术逻辑指令: add, sub, and, or, slt
- 取数/存数指令: lw, sw
- 控制流指令: beq, j

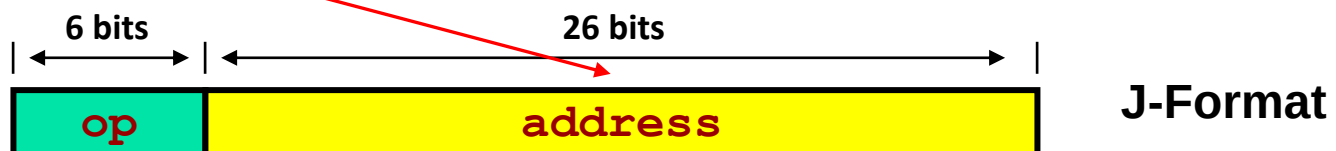
add \$t0, \$s1, \$s2



lw \$t0, 1002 (\$s2) beq \$t0, \$t1, Lable

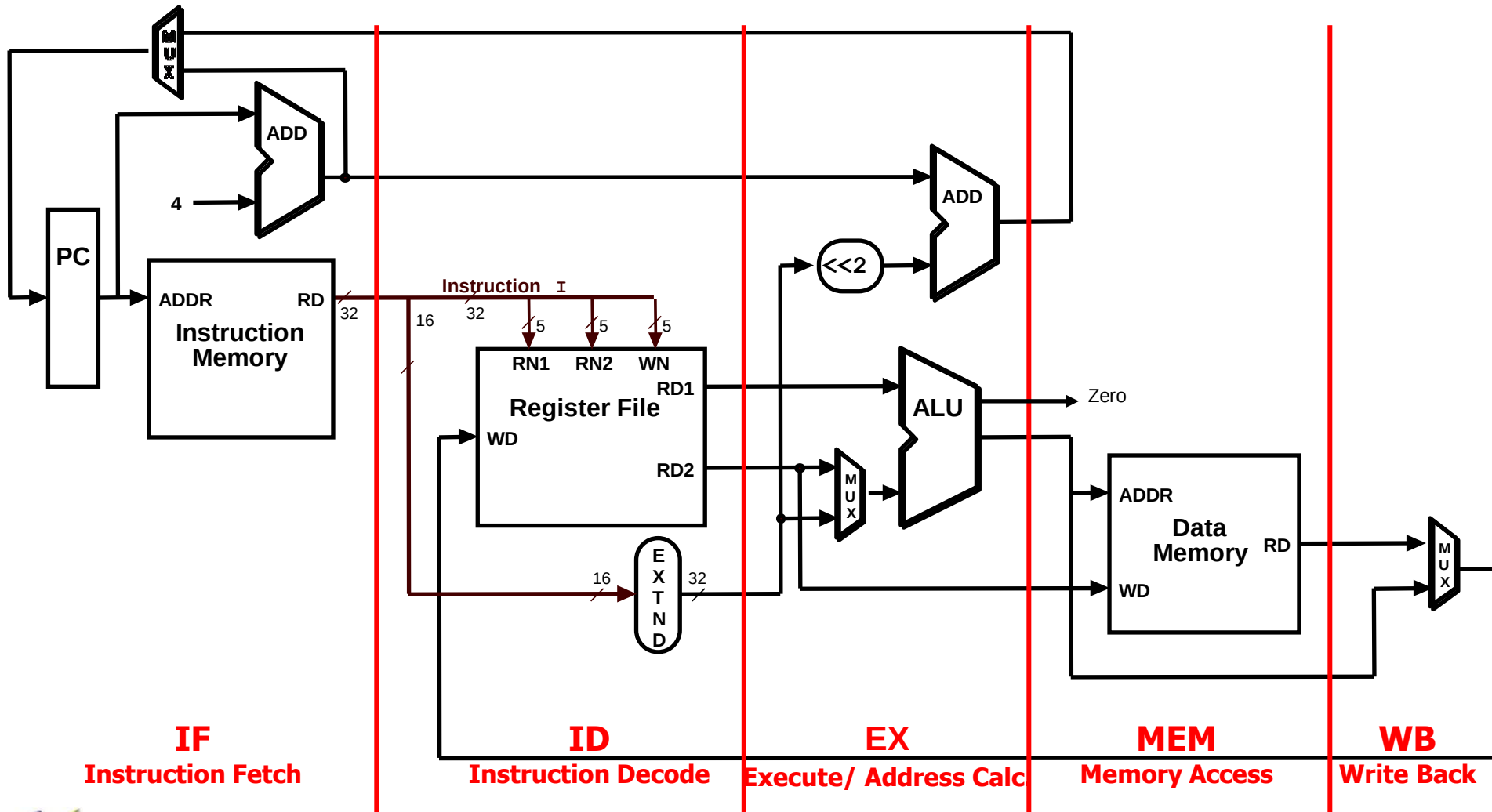


j Lable



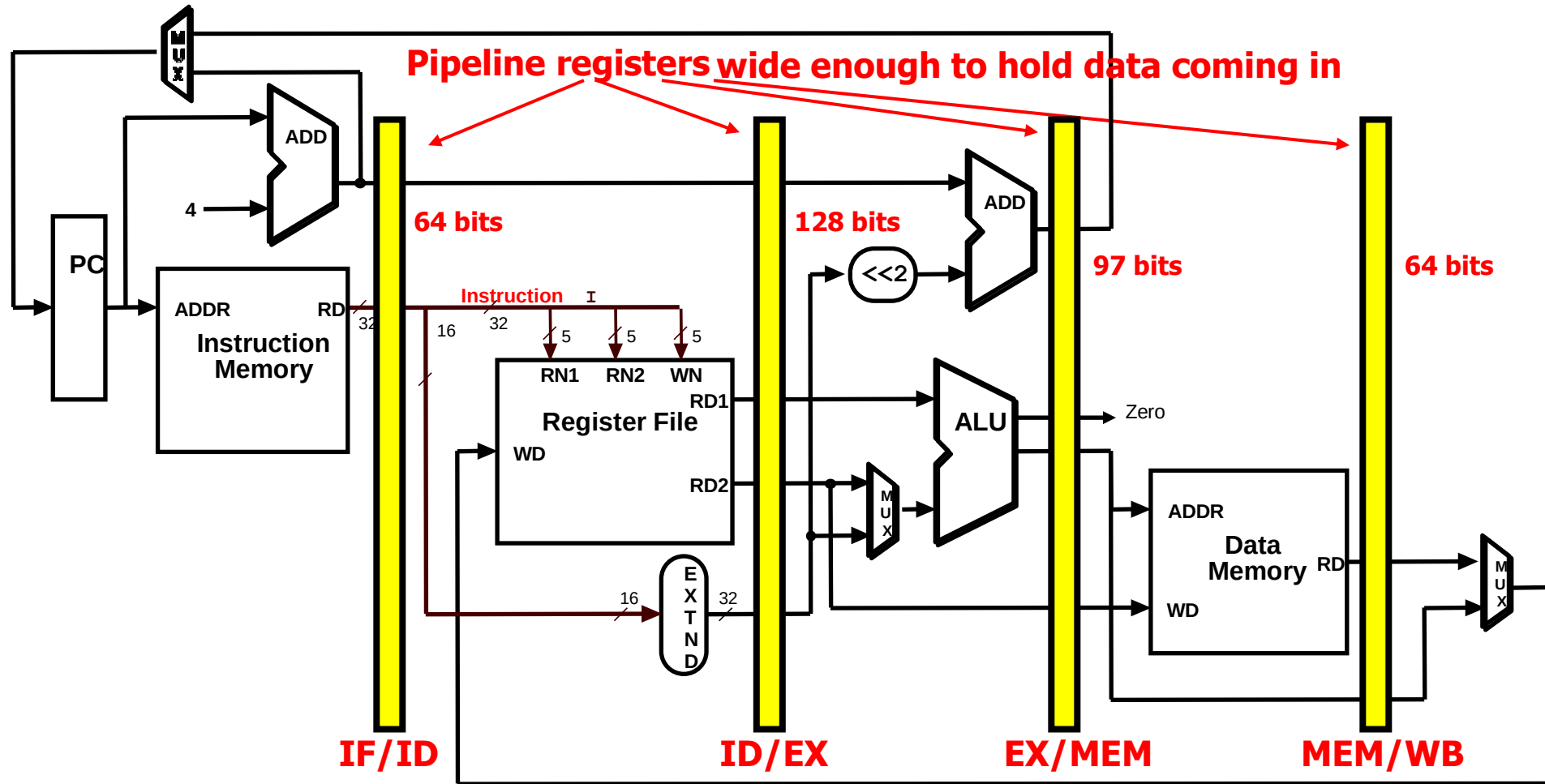


回顾 单周期MIPS数据通路图





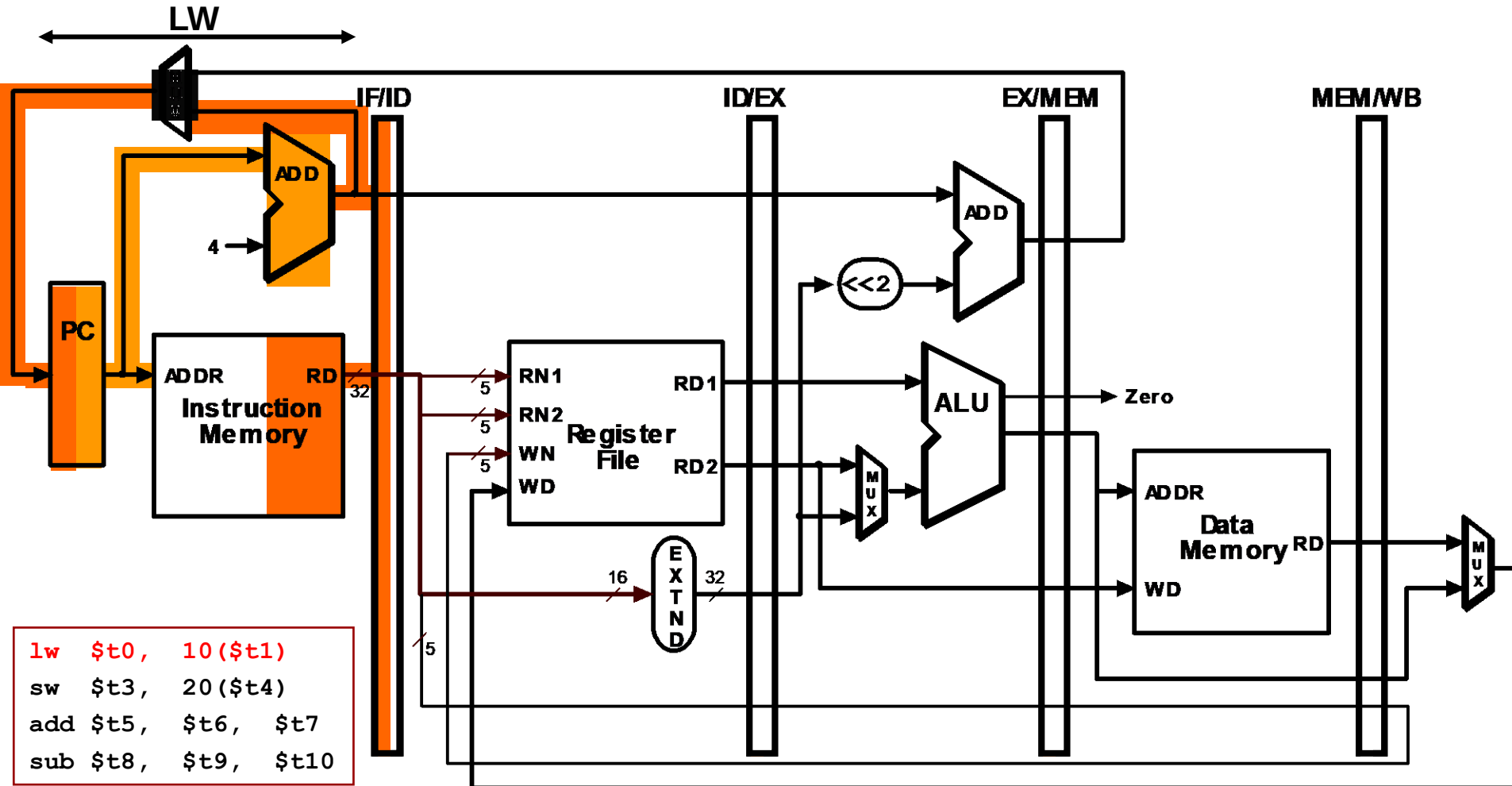
流水线MIPS数据通路图





流水线数据通路图

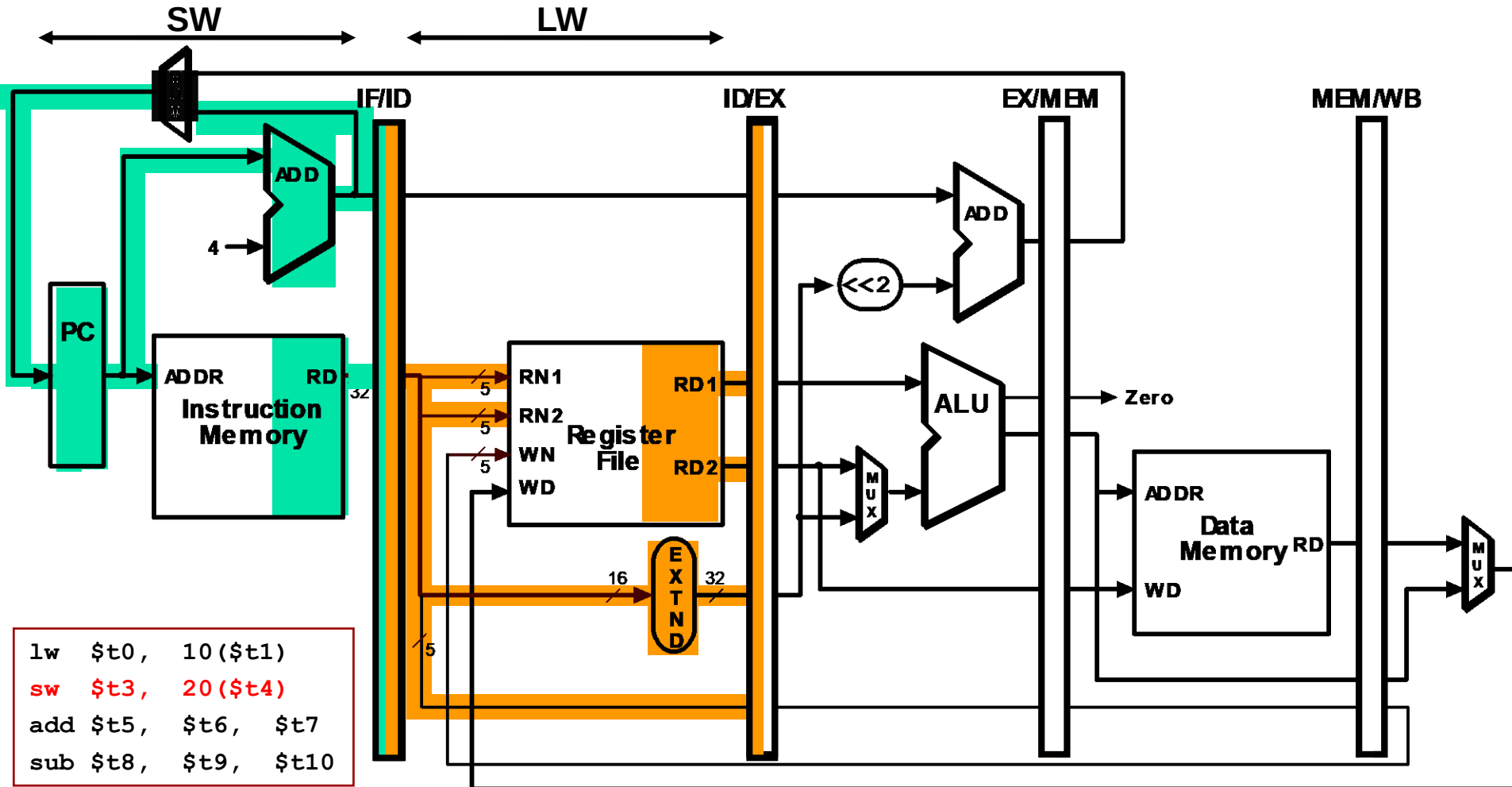
Clock 1





流水线数据通路图

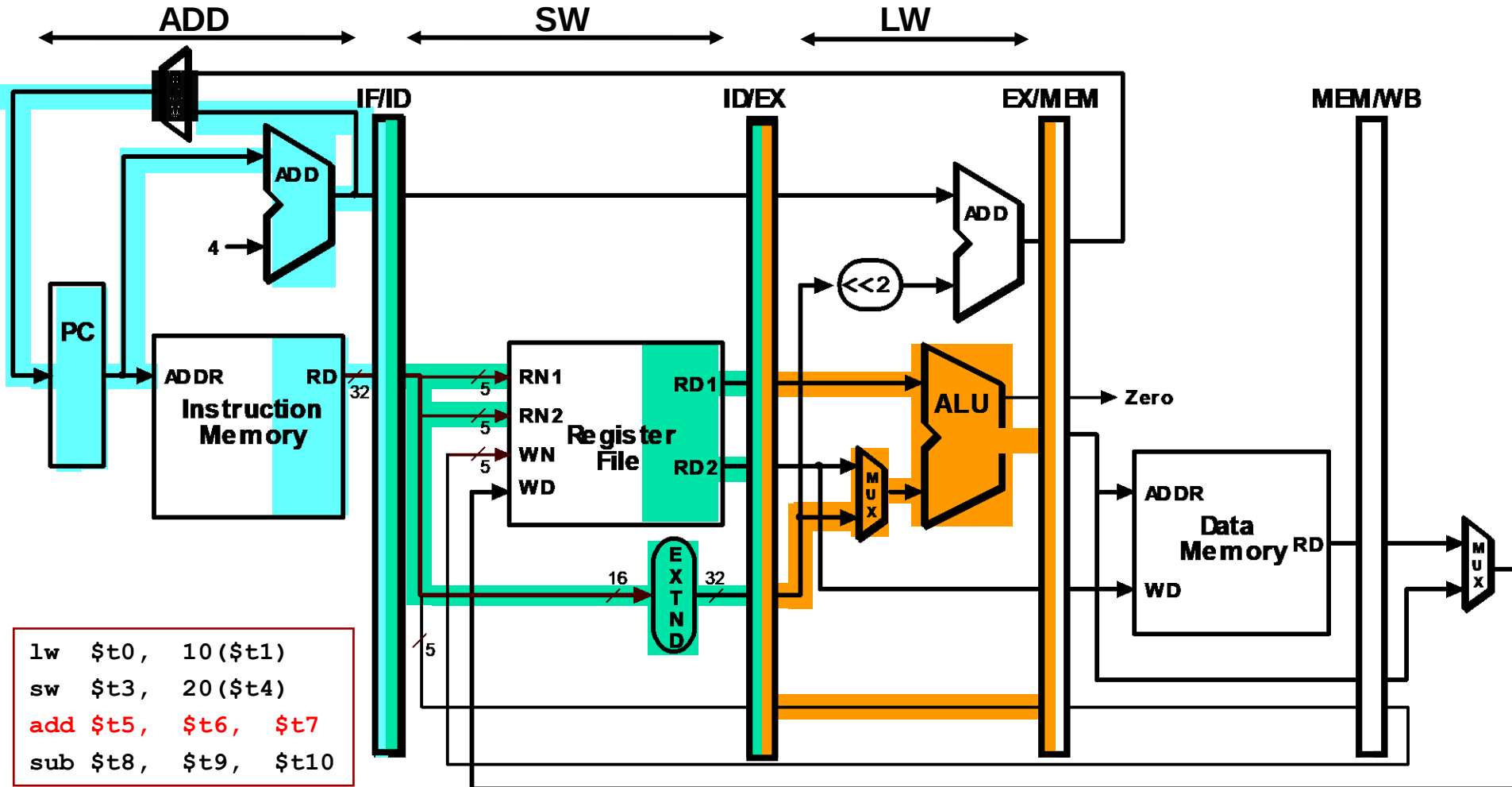
Clock 2





流水线数据通路图

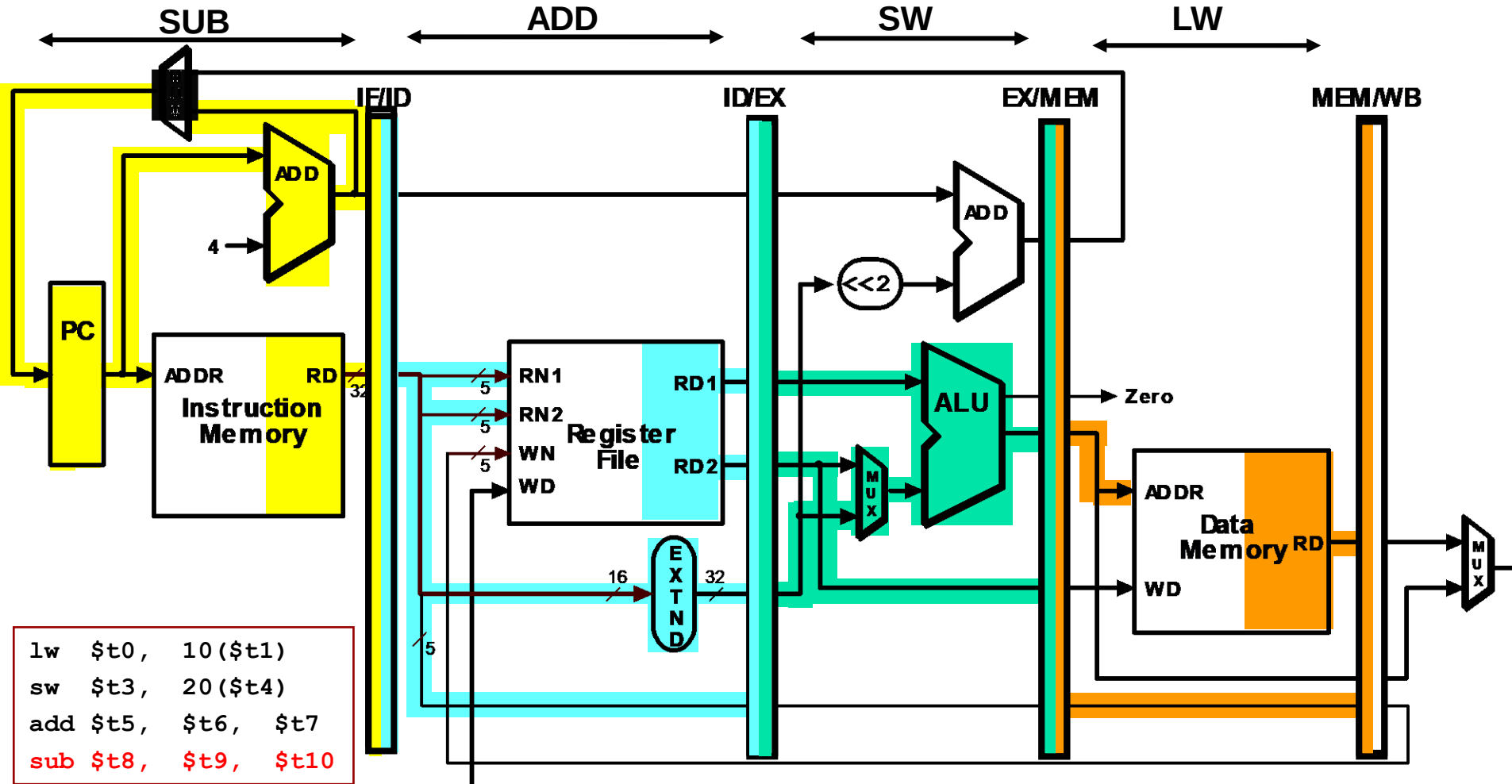
Clock 3





流水线数据通路图

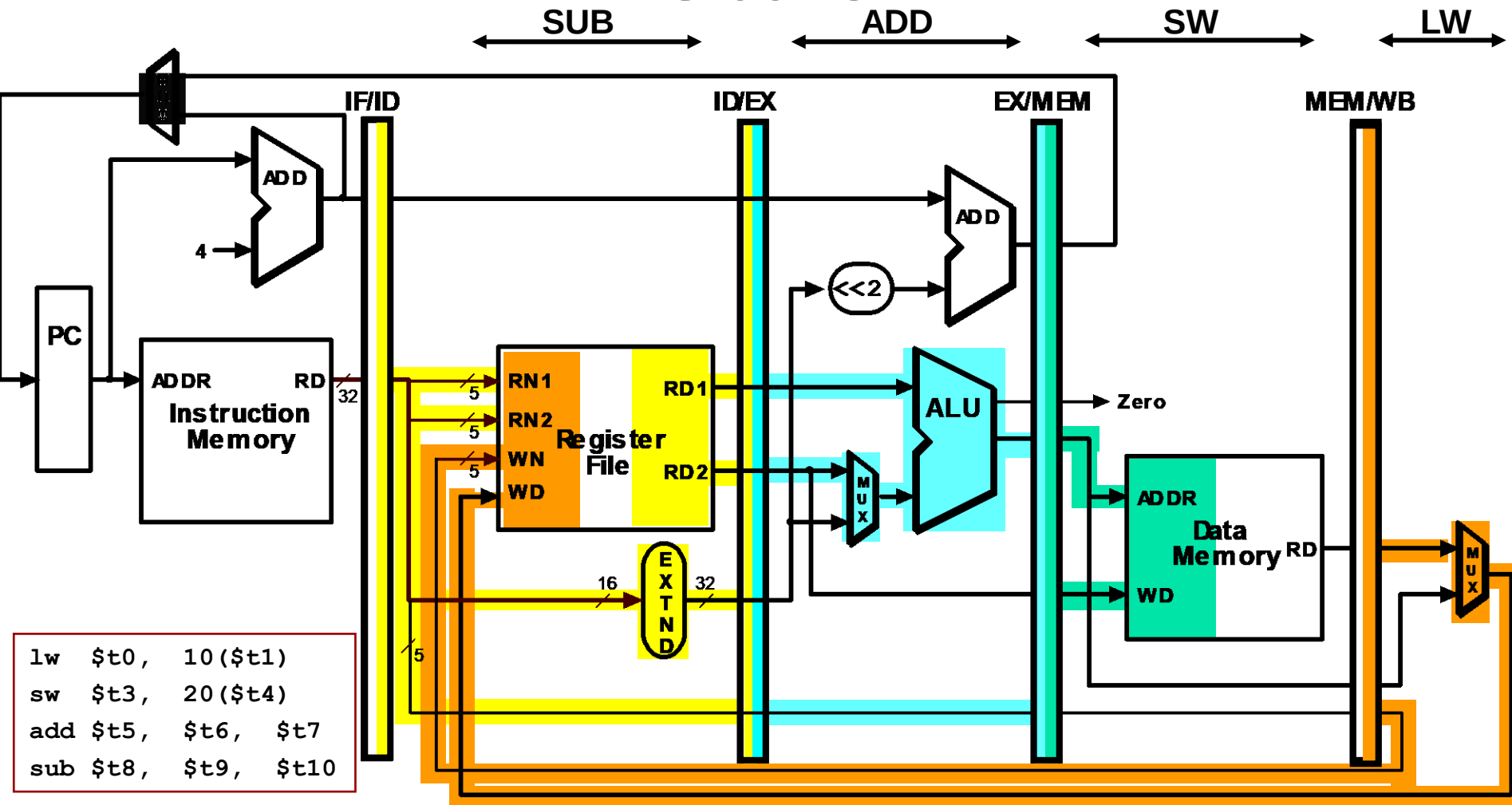
Clock 4





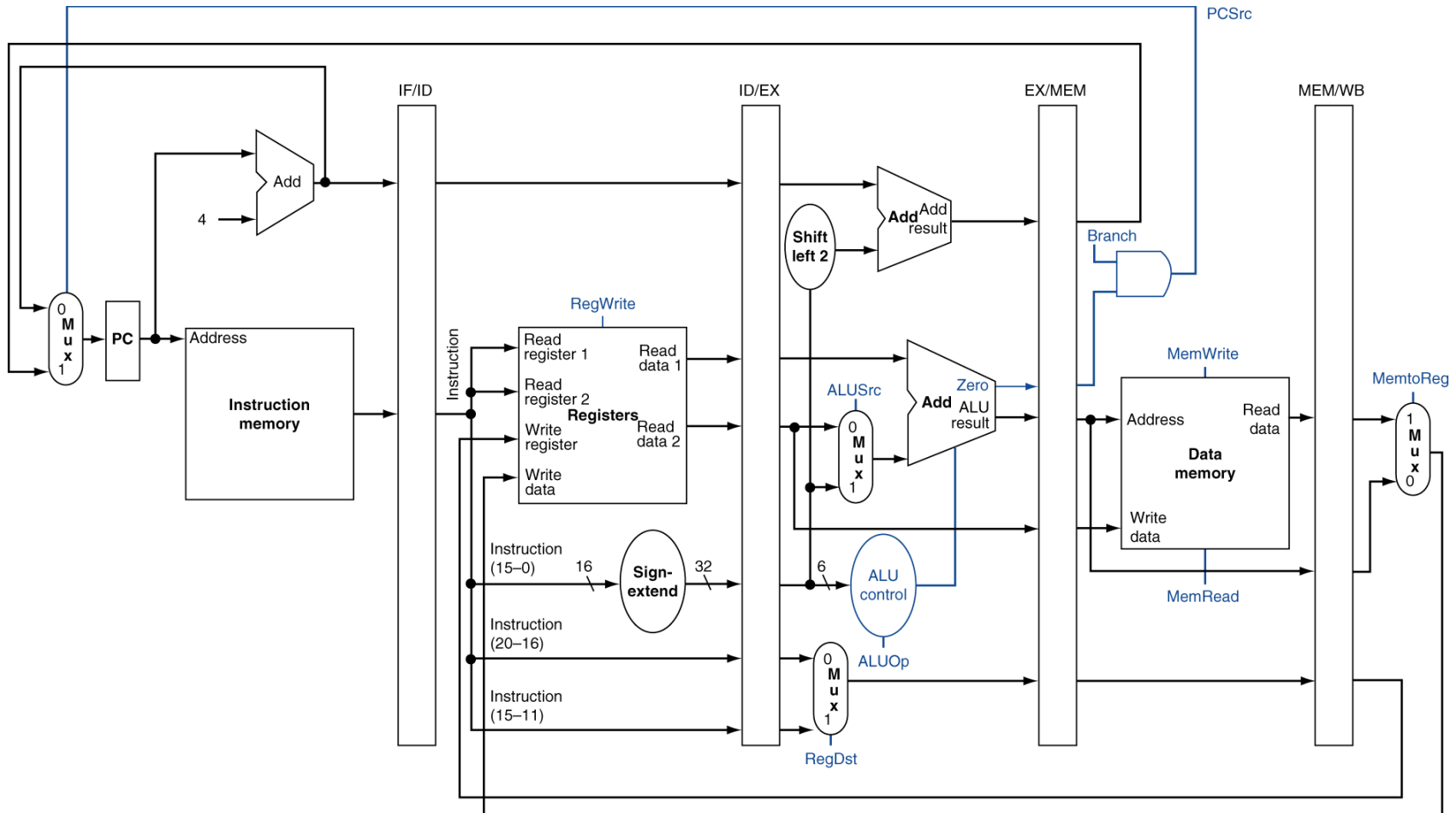
流水线数据通路图

Clock 5





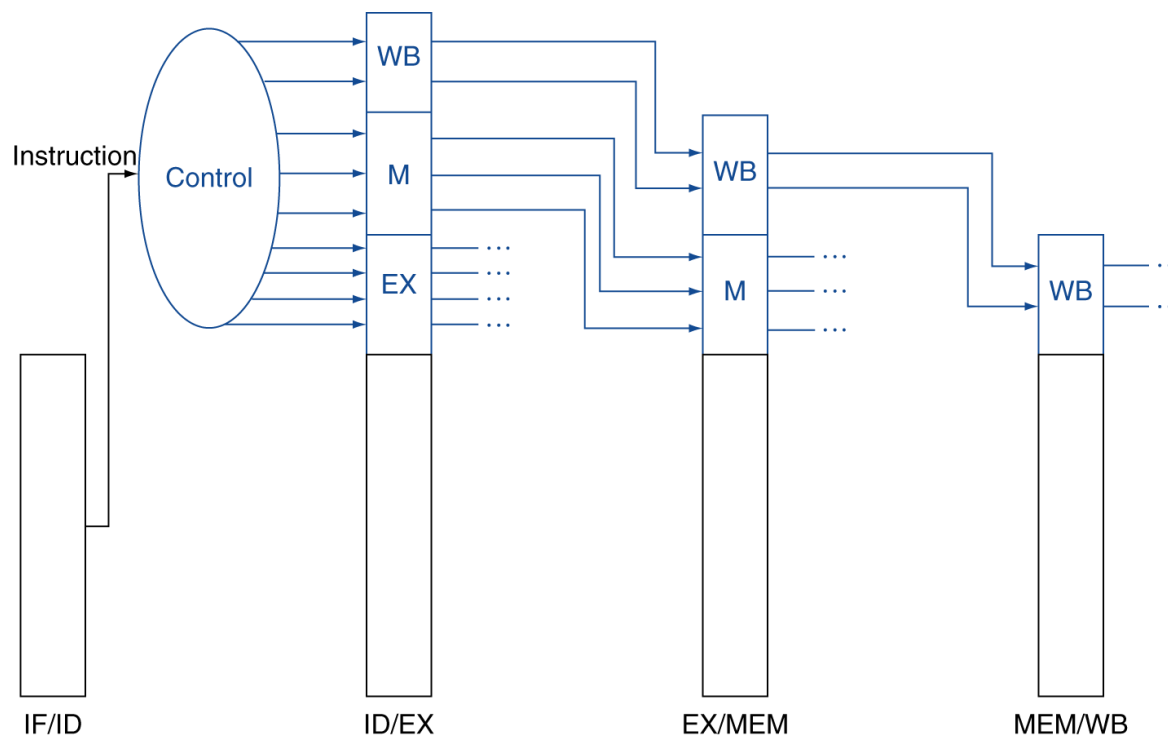
增加控制信号的流水线数据通路图



流水线的控制

■ 指令的操作码（31~26）生成控制信号

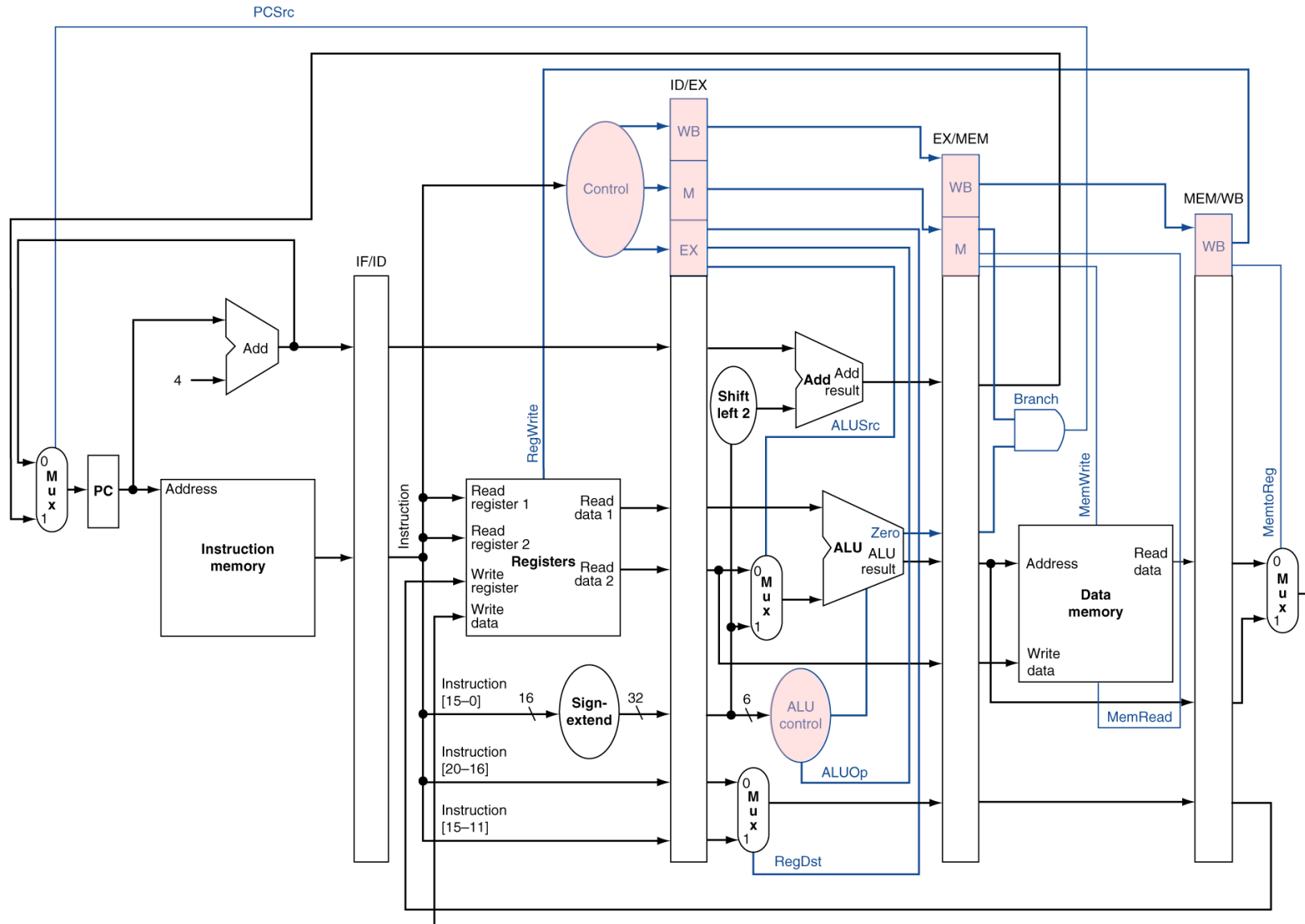
指令	执行/地址计算阶段的控制信号				存储器存取阶段的控制信号			写回阶段的控制信号	
	RegDst	ALUOp1	ALUOp0	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg
R 型	1	1	0	0	0	0	0	1	0
lw	0	0	0	1	0	1	0	1	1
sw	X	0	0	1	0	0	1	0	X
beq	X	0	1	0	1	0	0	0	X



与单周期数据通路的控制信号相同，只是把这9个控制信号按流水级进行了分组



流水线控制





5.7 RISC CPU



RISC处理器特点

- 1980年加州大学伯克利分校David Patterson和斯坦福大学的Hennessy教授各自设计了基于RISC思想处理器，随后推出了非常成功的商业芯片

- RISC的三个要素

- ◆ 一个有限的简单的指令集
- ◆ CPU具有大量的通用寄存器
- ◆ 强调对指令流水线的优化

- SUN, SPARC(1987)
- MIPS, SGI:MIPS(1986)
- HP, PA-RISC,
- IBM与Motorola, PowerPC
- DEC, Compaq, Alpha AXP
- IBM RS6000(1990) first Superscalar RISC
- AEM
- RISC-V





CISC与RISC的主要特征对比

比较内容	CISC	早期RISC
指令系统	复杂、庞大	简单、精简
指令数目	一般大于200	一般小于100
指令格式	一般大于4	一般小于4
寻址方式	一般大于4	一般小于4
指令字长	不固定, x86-64 1~15B	等长
可访存指令	不加限制	只有取数/存数指令
各种指令使用频率	相差很大	相差不大
各种指令执行时间	相差很大	大多数在一个周期内完成
优化编译实现	很难	较容易
程序源代码长度	很短	较长
控制器实现方式	绝大多数为微程序控制	绝大多数为硬布线控制
过程调用	栈密集的过程链接	寄存器密集的过程链接



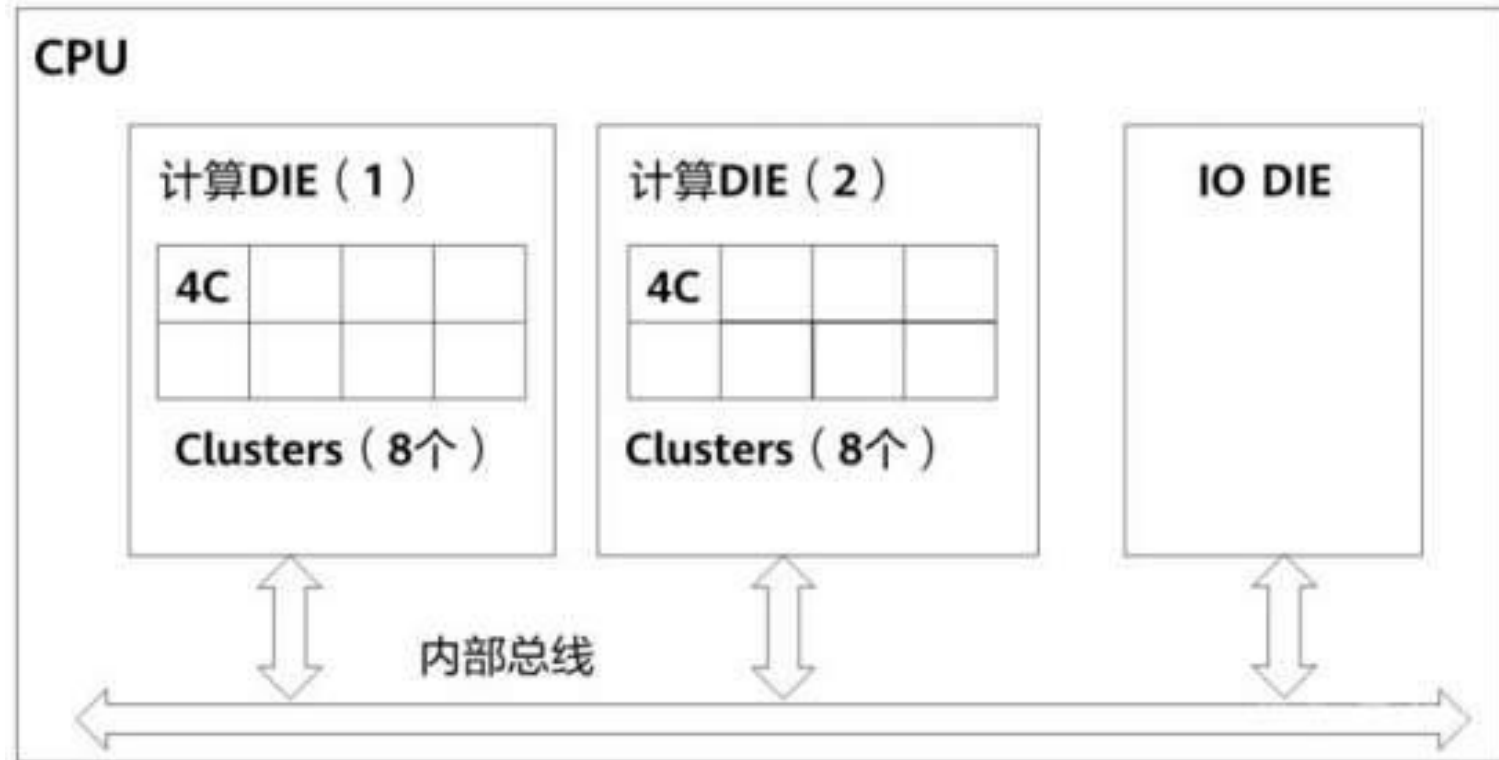


实例 鲲鹏处理器



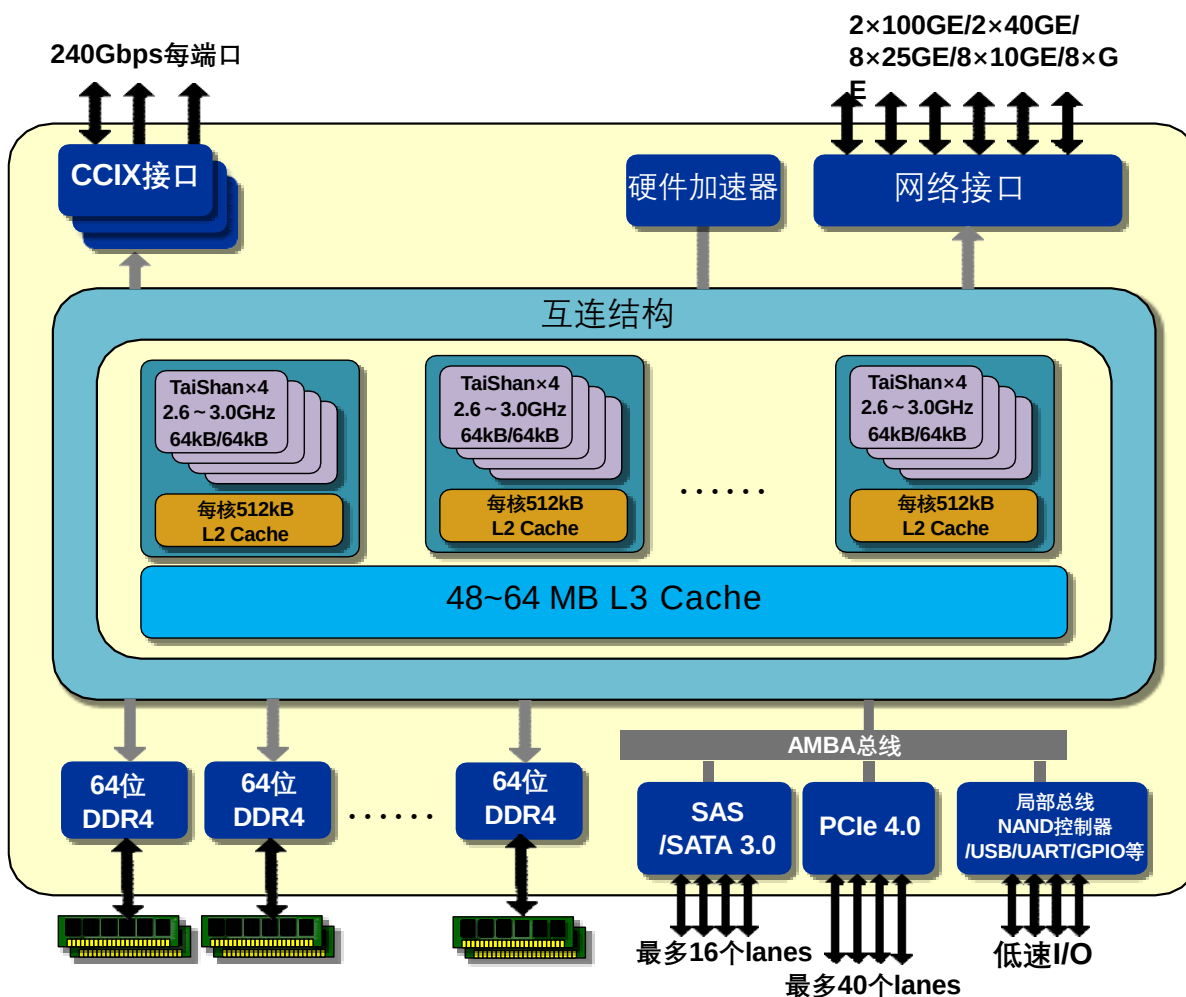


鲲鹏920处理器芯片架构

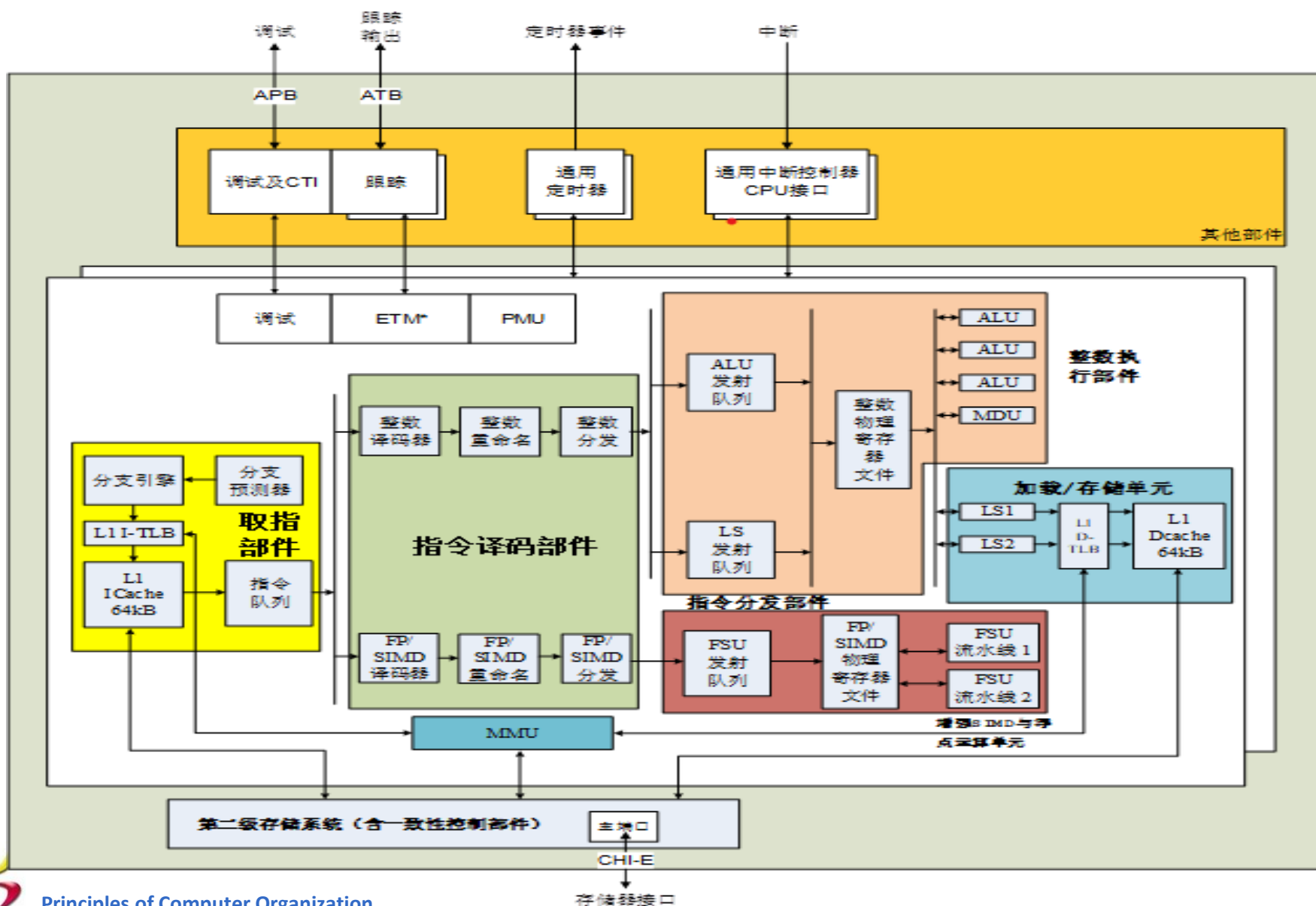


鲲鹏920处理器芯片的基本组成

- 由一致性互连总线包围起来的处理器内核
 - ◆ 最多可以配置64个TaiShan V110处理器内核
 - ◆ 集成多种系统控制器和核心外设控制器



TaiShanV110处理器内核的功能结构

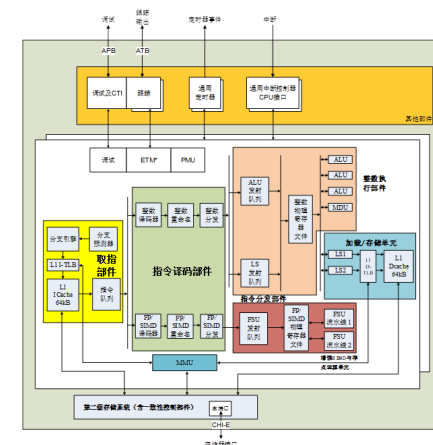




TaiShan V110处理器内核的功能结构

- Instruction fetch部件
- Instruction Decode部件
- Instruction Dispatch（指令分发）部件
- Integer Execute部件
- Load/Store Unit
- L2 Memory System
- Advanced SIMD and Floating-point Unit
- GIC CPU Interface
- Generic Timer
- PMU及Debug and Trace部件

多发射、乱序执行、
优化分支预测……





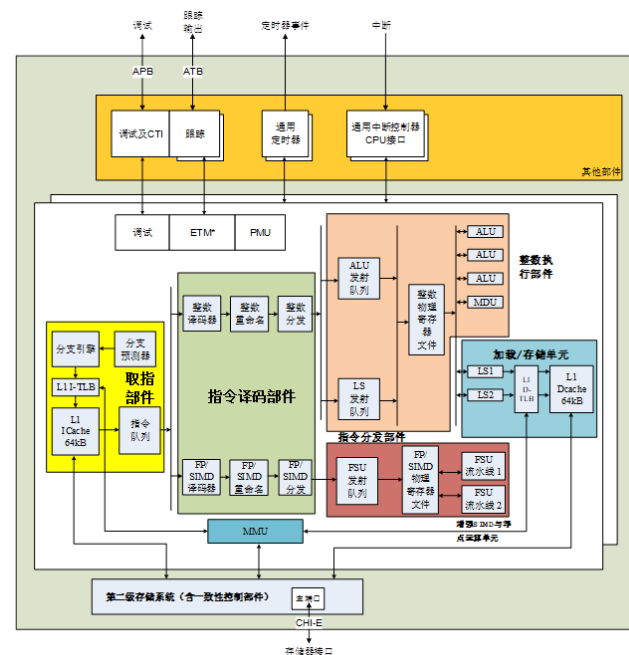
TaiShan V110处理器内核的功能结构

■ 取指部件

- ◆ 从L1指令Cache取出指令并向指令译码部件发送指令
 - 每个周期最多发送4条指令
- ◆ 支持动态分支预测和静态分支预测
 - 两级动态预测器内置分支目标缓冲器（BTB）
- ◆ 集成64KB的4路组相联L1指令Cache，行大小64B
- ◆ 32表项全相联L1指令TLB

■ 指令译码部件

- ◆ A64指令集译码
- ◆ 寄存器重命名
 - 消除WAW和WAR冒险
 - 支持指令的乱序执行

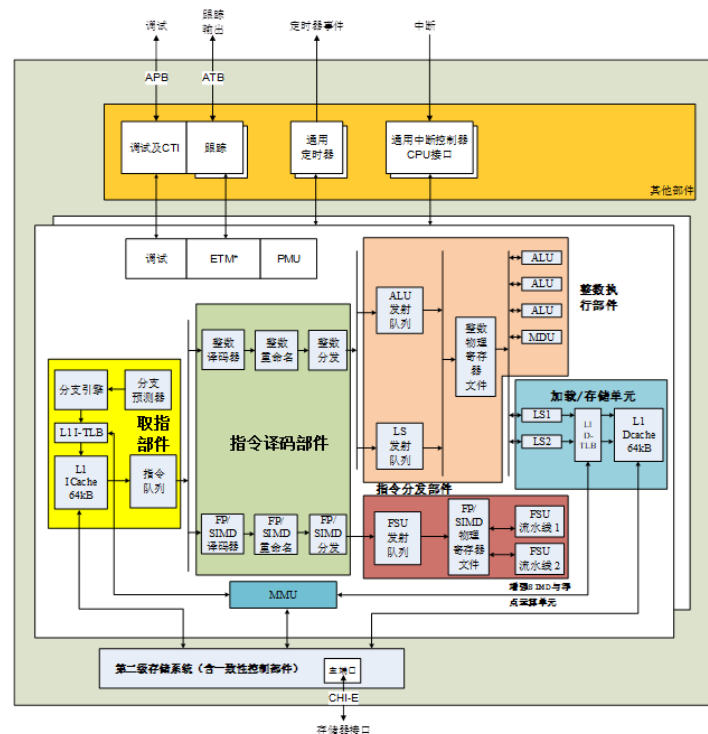


■ 指令分发部件 (Instruction Dispatch)

- ◆ 控制译码后的指令被分发至执行单元的指令流水线的~~时间~~及返回结果被放弃的时间
- ◆ 包含通用寄存器文件、增强SIMD及浮点寄存器文件和AArch64状态下的系统寄存器

■ 整数执行部件

- ◆ 3条算术逻辑运算单元（ALU）流水线
- ◆ 1条整数乘除运算单元（MDU）流水线
- ◆ 交互式整数除法硬件电路
- ◆ 分支与指令条件码解析逻辑
- ◆ 结果转发与比较器逻辑电路



*ETM:Embedded Trace Macrocell, 嵌入式跟踪宏单元

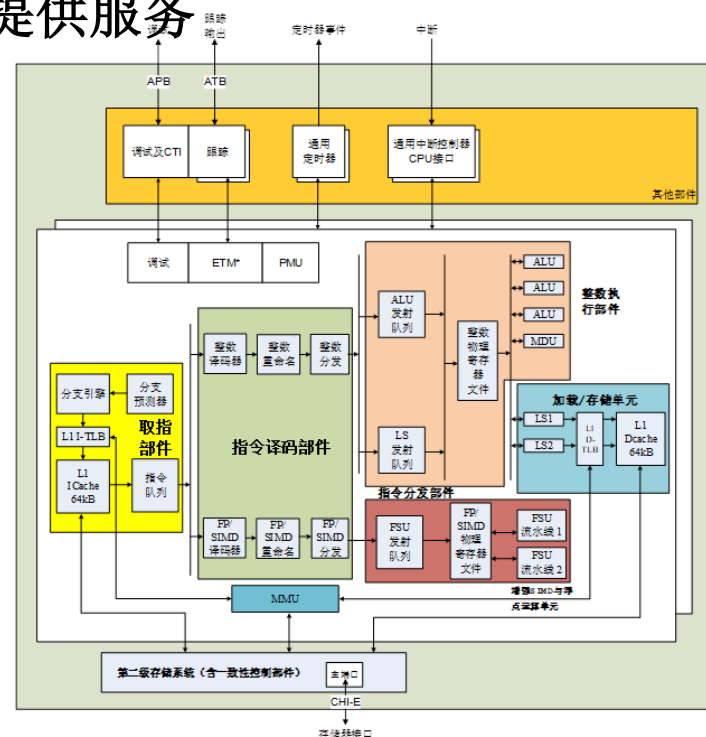
TaiShan V110处理器内核的功能结构

■ Load/Store (LS) 单元

- ◆ 执行加载和存储指令
- ◆ 包含L1数据Cache相关部件
 - L1 D Cache容量64KB，4路组相联，Cache行大小64B
 - 32表项全相联L1数据TLB
- ◆ 为来自L2存储系统的存储器一致性请求提供服务

■ 第二级存储系统

- ◆ 512KB的L2 Cache，8路组相联
- ◆ L1 Cache 标记RAM副本，用于处理监听请求
- ◆ 每个处理器包含1152表项的4路组相联L2 TLB
- ◆ 集成自动硬件预取器



TaiShan V110处理器内核的功能结构

■ 浮点和SIMD运算单元（FSU）

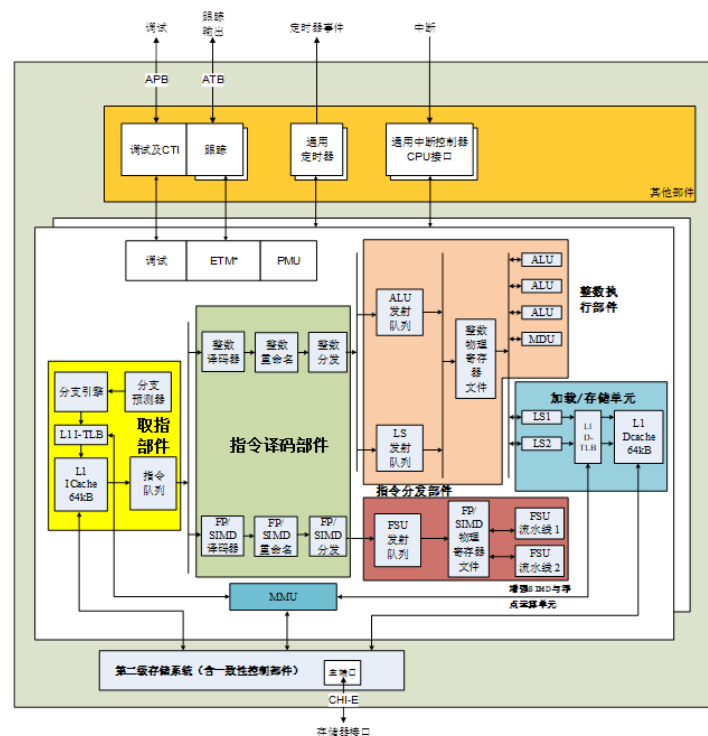
- ◆ 支持ARMv8-A架构的增强SIMD与浮点运算类指令
- ◆ 支持可选的加密引擎

■ 通用中断控制器CPU接口

- ◆ 负责向处理器发送中断请求

■ 通用定时器

- ◆ 为事件调度提供支持，并可以触发中断



*ETM: Embedded Trace Macrocell, 嵌入式跟踪单元

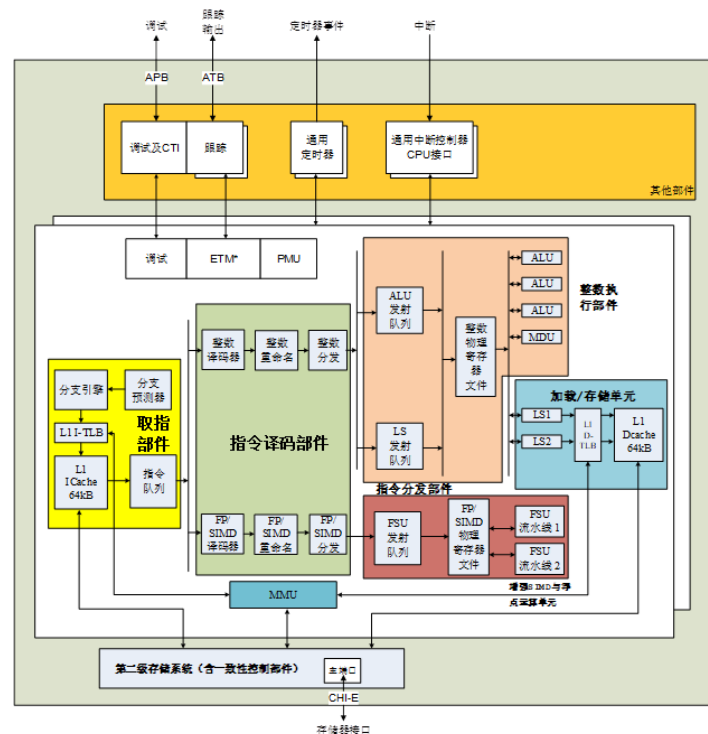
TaiShan V110处理器内核的功能结构

■ PMU（性能监视单元）

- ◆ 为调优系统性能提供支持
- ◆ 监视ARMv8-A架构定义的事件以及鲲鹏自定义的事件
- ◆ 软件可以通过PMU获取PMU事件的性能信息

■ 调试与跟踪部件

- ◆ 支持ARMv8的调试架构
 - 通过AMBA APB从接口访问调试寄存器



*ETM: Embedded Trace Macrocell, 嵌入式跟踪单元



ARM处理器体系结构：指令流水线

■ ARM流水线的执行顺序：

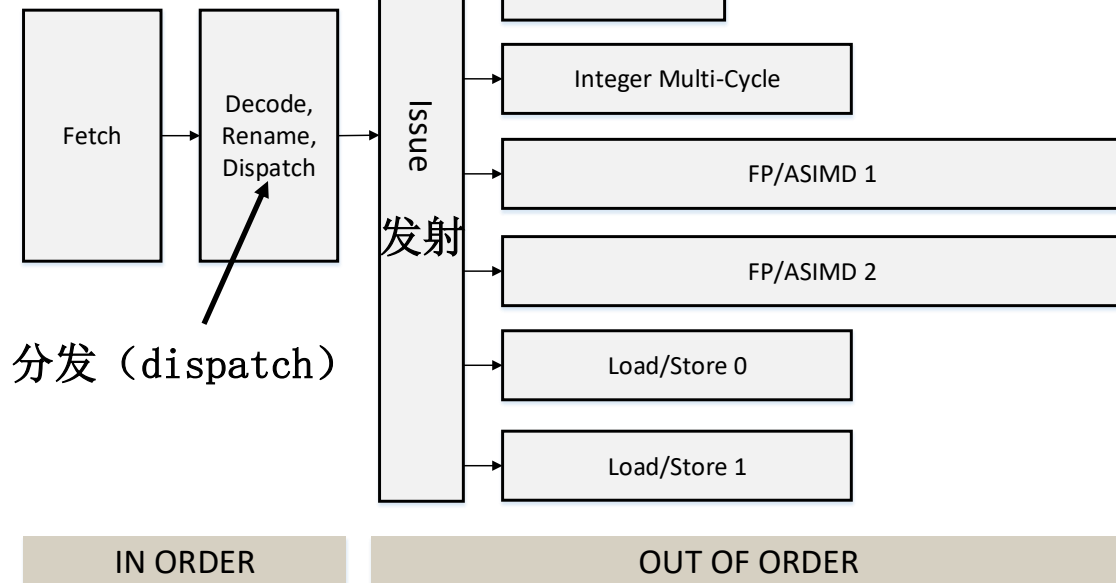
- ◆ 取指令（Fetch）：从存储器读取指令
- ◆ 译码（Decode）：译码以鉴别属于哪一条指令
- ◆ 执行（Execute）：将操作数进行组合以得到结果或存储器地址
- ◆ 缓冲/数据（Buffer/data）：如果需要，则访问存储器以存储数据
- ◆ 写回（Write-back）：将结果写回到寄存器组中





鲲鹏流水线技术 (1)

分发：指令译码后被派发到不同的运算单元等待队列中的过程



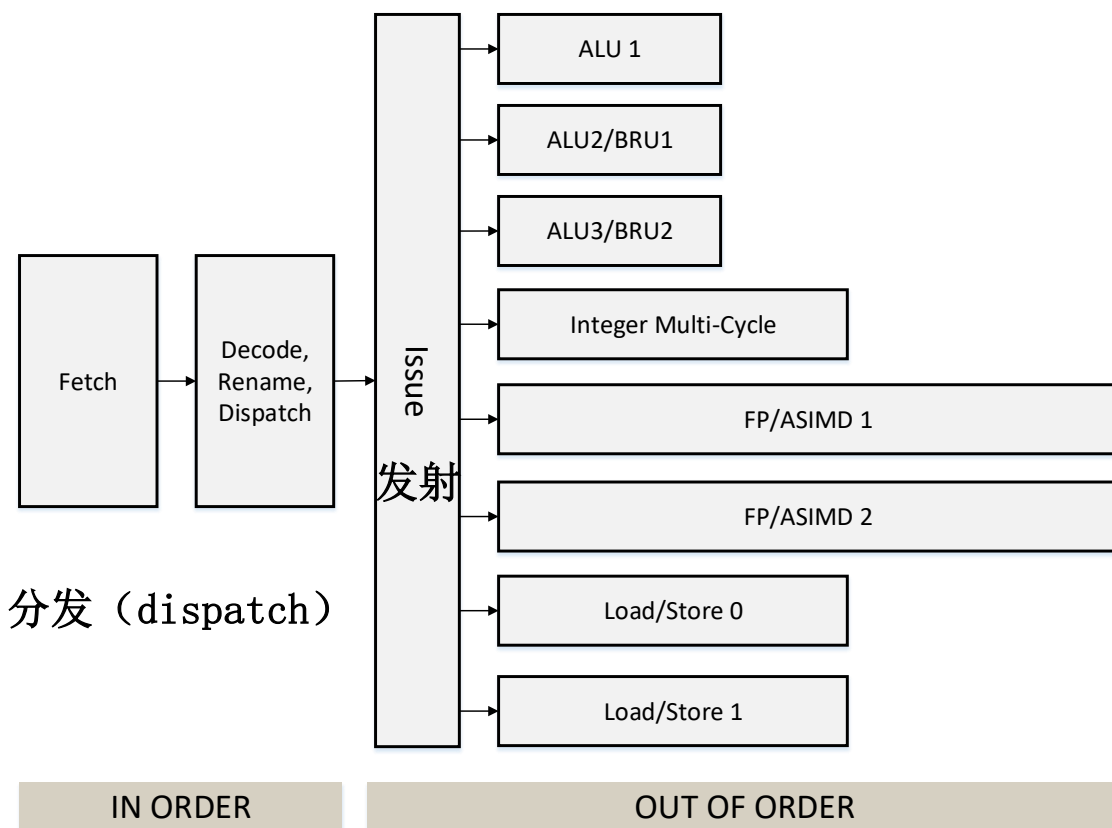
发射 (Issue)：指令从运算单元等待队列中发射到运算单元开始执行的过程

- 取指单元：每拍最多可提供 $8 \times 4 = 32$ 字节指令供译码
- 整型运算单元：支持每拍3条ALU运算（含2条跳转）及1条乘除运算
- 浮点及SIMD运算单元：支持每拍2条ARM Neon 128位浮点及SIMD运算
- 访存单元：支持每拍2条读或写访存操作
 - ◆ 读操作最快4拍完成
 - ◆ 每拍访存带宽为 2×128 位读及 1×128 位写





鲲鹏流水线技术 (2)



乱序执行

- ◆ 不按程序规定的顺序执行指令
- ◆ 根据内部功能部件的空闲状态动态分发执行指令
- ◆ 指令结束的顺序按照原有程序规定的顺序
- ◆ 处理器内部功能部件并行运行，避免不必要的阻塞
 - 处理器硬件分析影响执行结果的指令，避免出现显式的数据冒险和控制冒险的乱序
 - 特殊情况下的读写乱序可能影响程序执行结果，需要软件甄别





本章小结

- CPU模型
- 指令周期、指令执行过程
- 时序产生器和控制方式
- 微程序控制器
 - ◆ 微命令、微操作、微指令、微程序
 - ◆ 微程序控制器
 - ◆ 微程序设计技术
- 硬布线控制器
- 流水线CPU
 - ◆ 时空图、标量流水线、超标量流水线、加速比
 - ◆ 流水线冒险





例5

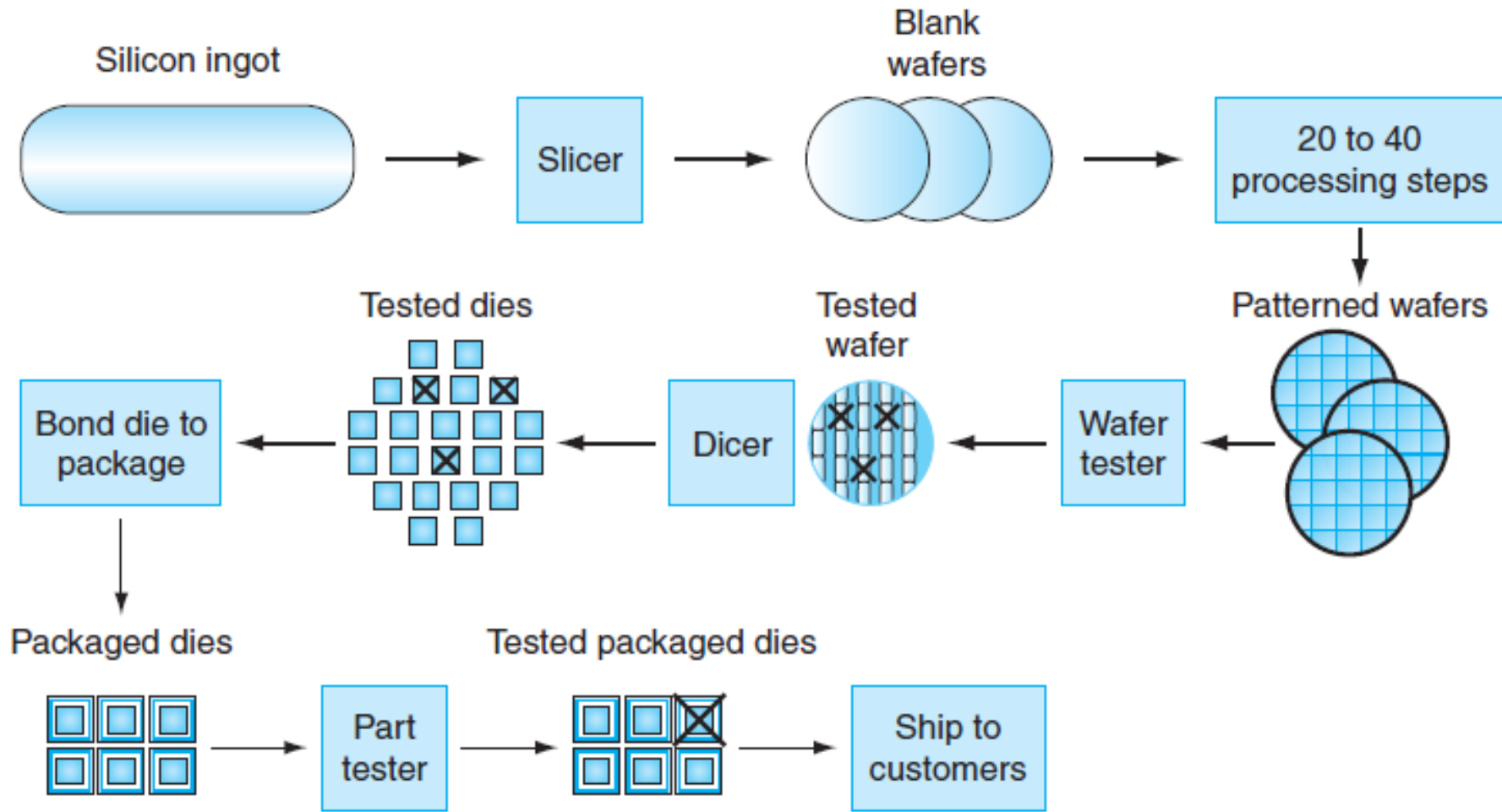
■ 教材第180页例5





知识拓展

芯片制造过程





课间休息

